esprit

Se former autrement

2025 - 2026

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY : INFORMATION TECHNOLOGY

CANvisionNative

A diagnostic and validation platform for the CAN traffic of Battery Management Systems

Prepared by: Mohamed Mootaz Ben Kraiem

Academic Supervisor: Mrs Nadia Boulifa

Corporate Supervisor: Mr Hamda Guizani

Capgemini engineering

cdio

EUR-ACE

Cti

CONFÉRENCE DES GRANDES ÉCOLES

Signatures

Corporate supervisor	**Academic supervisor**
Mr Hamda Guizani Capgemini Engineering	Mrs Nadia Boulifa ESPRIT
Signature and stamp	Signature

Dedication

To my dear family,

To my mother, whose strength was never loud, yet never failed me. Your patience, your sacrifices, and the quiet dignity with which you carried what others could not see have shaped more of me than words could ever account for. Whatever steadiness I possess, I owe in no small measure to you.

To my father, for supporting my studies and for playing a part in the road that led me here.

To my brothers, whose presence has remained a constant thread through the years: for the shared laughter, the disagreements, the silence, and the simple certainty that some bonds require no explanation.

To someone dear, who understood without demanding an explanation, who waited without complaint, and whose presence made difficult distances seem less so. Some debts are not measured in what is given, but in what is quietly endured beside us. This is for you.

And finally, to myself.

For having walked through doubt without allowing it to become a verdict, and for refusing to mistake a difficult chapter for the conclusion of the story.

I have been my own harshest judge. No longer.

The road ahead remains unwritten. Good. I have never required certainty to begin.

This is the first step of a promise I made to myself, one I intend to keep.



Acknowledgements

I would like to thank my academic supervisor at ESPRIT, Mrs Nadia Boulifa, for following this work from the beginning, for the time given to my questions, and for the advice that shaped this report.

I also thank my supervisor at Capgemini Engineering, Mr Hamda Guizani, for welcoming me into the Battery Management Systems team, for the trust placed in me during this work and study program, and for the guidance that helped me keep the project close to the real needs of the team. My thanks also go to the members of the Automotive division of Capgemini Tunisia for their availability and their help.

I thank my mother, for her love and for every sacrifice she made so that I could reach this point; my father, for supporting my studies; my brothers and my family, for their encouragement; and someone dear, for standing by my side every step of the way. To everyone who supported me, directly or indirectly, thank you.

Finally, I thank the members of the jury for agreeing to evaluate this work.

Contents

Dedication

Acknowledgements

General Introduction ix

1 General Framework 1
- 1.1 Introduction . . . 1
- 1.2 Host organization . . . 1
  - 1.2.1 Presentation . . . 1
  - 1.2.2 Department and work context . . . 1
- 1.3 Project context . . . 2
- 1.4 Study of existing solutions . . . 2
  - 1.4.1 CAN analysers . . . 2
  - 1.4.2 Intrusion detection research . . . 3
  - 1.4.3 Viewers and diagnostic scanners . . . 3
  - 1.4.4 Synthesis . . . 3
- 1.5 Problem statement . . . 4
- 1.6 Proposed solution . . . 4
- 1.7 Objectives . . . 5
  - 1.7.1 General objective . . . 5
  - 1.7.2 Specific objectives . . . 5
- 1.8 Development methodology . . . 6
  - 1.8.1 Comparison: Waterfall, Scrum and the Unified Process . . . 6
  - 1.8.2 Principles and phases of the Unified Process . . . 7
  - 1.8.3 The Unified Process applied to CANvisionNative . . . 7
  - 1.8.4 Modelling language and tools . . . 8
- 1.9 Conclusion . . . 9

2 Inception: Requirements Capture 10
- 2.1 Introduction . . . 10
- 2.2 Actors . . . 10
- 2.3 Functional requirements . . . 11
- 2.4 Non functional requirements . . . 12
- 2.5 Use case model . . . 13
  - 2.5.1 Global use case diagram . . . 13
  - 2.5.2 Textual descriptions . . . 15
  - 2.5.3 Activity diagram . . . 18
- 2.6 Initial risk list . . . 19

2.7 Project plan by phase and iteration . . . 20
2.8 Conclusion . . . 20

3 Elaboration: Analysis and Architecture 21
3.1 Introduction . . . 21
3.2 Technical choices . . . 21
3.3 Overall architecture . . . 23
3.3.1 Logical view in layers . . . 23
3.3.2 Components . . . 24
3.3.3 Client architecture (MVVM) . . . 25
3.3.4 Deployment . . . 26
3.4 Analysis model . . . 27
3.4.1 Domain model . . . 27
3.4.2 Class diagrams . . . 28
3.4.3 Data model . . . 30
3.5 Dynamic view . . . 30
3.5.1 Life cycle of a replay . . . 31
3.5.2 Explanation of an alert . . . 32
3.6 Detection architecture . . . 33
3.6.1 Frame by frame path (live, simulation, replay) . . . 34
3.6.2 Offline analysis path . . . 34
3.7 Data and machine learning problem framing . . . 35
3.7.1 Problem definition . . . 35
3.7.2 Dataset files . . . 35
3.7.3 Supervised or unsupervised learning . . . 36
3.7.4 Candidate algorithms . . . 36
3.7.5 Choice of Isolation Forest . . . 36
3.8 Architecture baseline . . . 37
3.9 Conclusion . . . 37

4 Construction: Iterations 38
4.1 Introduction . . . 38
4.2 Iteration planning . . . 38
4.3 Iteration 1: first detection pipeline and client . . . 39
4.3.1 Objectives . . . 39
4.3.2 Use cases addressed . . . 39
4.3.3 Design: a first model based on driving states . . . 39
4.3.4 Implementation . . . 40
4.3.4.1 Parser and common frame representation . . . 40
4.3.4.2 Replay engine . . . 40
4.3.4.3 Vehicle profiles . . . 41
4.3.4.4 Explanation module . . . 41
4.3.4.5 Client and exports . . . 41
4.3.5 Tests . . . 41
4.3.6 Assessment . . . 42
4.4 Iteration 2: validation and correction on the Kia EV6 . . . 42
4.4.1 Objectives . . . 42
4.4.2 Use cases addressed . . . 42

4.4.3 Design of the validation . . . 43
4.4.4 Implementation: audit and corrections . . . 43
4.4.4.1 Interactive interface audit . . . 43
4.4.4.2 Conversion of the MF4 file . . . 44
4.4.4.3 Root cause analysis of the alerts . . . 44
4.4.4.4 Fix 1: vehicle model selection . . . 44
4.4.4.5 Fix 2: driving state classification . . . 45
4.4.4.6 Fix 3: attribution of the reason . . . 45
4.4.4.7 Fix 4: one parser for replay . . . 46
4.4.4.8 Fix 5: polling timer left stopped . . . 46
4.4.4.9 Fix 6: stale explanation cache . . . 47
4.4.5 Tests: measurements before and after the corrections . . . 48
4.4.6 Assessment . . . 49
4.5 Iteration 3: hardware layer and bench rig . . . 50
4.5.1 Objectives . . . 50
4.5.2 Use cases addressed . . . 50
4.5.3 Design . . . 50
4.5.4 Implementation . . . 50
4.5.5 Tests . . . 51
4.5.6 Assessment . . . 51
4.6 Iteration 4: model rebuild . . . 52
4.6.1 Objectives . . . 52
4.6.2 Use cases addressed . . . 52
4.6.3 Design . . . 52
4.6.3.1 Final design: one model per CAN identifier . . . 52
4.6.3.2 Training and promotion pipeline . . . 53
4.6.3.3 Model status . . . 55
4.6.4 Implementation . . . 56
4.6.5 Tests . . . 57
4.6.5.1 Evaluation protocol . . . 57
4.6.5.2 Holdout results . . . 57
4.6.5.3 Models we did not put into production . . . 58
4.6.5.4 Consistency checks . . . 58
4.6.5.5 Speed of the final models . . . 58
4.6.5.6 Comparison with the other candidate algorithms . . . 59
4.6.6 Assessment . . . 61
4.7 Conclusion . . . 61

5 Transition: Final Release and Validation 62
5.1 Introduction . . . 62
5.2 Final application tour . . . 62
5.3 Test strategy and results . . . 68
5.3.1 Test plan . . . 68
5.3.2 Automated tests . . . 69
5.3.3 End to end test of the release . . . 69
5.3.4 Performance . . . 70
5.4 Requirements traceability . . . 70
5.5 Deployment and documentation . . . 71

5.5.1 Installation 71
5.5.2 Model promotion 71
5.5.3 User guide 72
5.6 Stability scope and limitations 72
5.7 Conclusion 73

General Conclusion **74**

Bibliography **77**

Abstract **80**

Résumé **81**

List of Figures

1.1 Logo of Capgemini Engineering. . . . 1
1.2 Existing tools and diagnostic standards. . . . 2
1.3 Phases and iterations of the Unified Process in the project. . . . 8

2.1 Use case diagram of CANvisionNative. . . . 13
2.2 Activity diagram: importing, analysing and replaying a log. . . . 18
2.3 Project plan by phase and iteration, February to October 2026. . . . 20

3.1 Technologies used in the project. . . . 21
3.2 Overall layered architecture of CANvisionNative. . . . 23
3.3 Component diagram of the client and the backend. . . . 24
3.4 Frontend architecture of the client. . . . 25
3.5 Deployment diagram. . . . 26
3.6 Domain model of CANvisionNative. . . . 27
3.7 Simplified class diagram of the detection core of the backend. . . . 28
3.8 Class diagram of the desktop client. . . . 29
3.9 Data model of the SQLite store. . . . 30
3.10 Sequence diagram: life cycle of a replay. . . . 31
3.11 Sequence diagram: requesting the explanation of an alert. . . . 32
3.12 Detection pipeline of the backend. . . . 33
3.13 Sources of the recordings. . . . 35

4.1 Scoring path of the first design. . . . 39
4.2 Passes of the validation of the first pipeline. . . . 43
4.3 Alerts, saturated model score and replay throughput on the Kia EV6 recording, before and after the corrections. . . . 48
4.4 Driving state of the whole recording, before and after Fix 2. . . . 48
4.5 Severity and dominant layer of the alerts after Fix 3. . . . 49
4.6 Training pipeline, from the recordings of a vehicle to a model in production. . . . 54
4.7 State diagram: life cycle of a vehicle model. . . . 56
4.8 Pipeline of the algorithm comparison. . . . 59
4.9 Share of the holdout rows flagged by each algorithm. . . . 60
4.10 Training time and time to score one frame. . . . 60
4.11 Features used by the final Isolation Forests. . . . 61

5.1 Home screen: choice between a live session and an offline import. . . . 62
5.2 Dashboard after the analysis of the Jaguar I-PACE holdout recording. . . . 63
5.3 Dataset card of the Jaguar I-PACE holdout recording, open over the Dashboard. 64
5.4 Log Playback during the replay of the Jaguar I-PACE holdout recording. . . . 64

5.5 Anomaly Intel: alert list, signals of the selected alert and explanation panel. . . 65
5.6 First page of the PDF report of the Jaguar I-PACE analysis (339 alerts). 66
5.7 First rows of the exported CSV file of the same analysis. 66
5.8 Telemetry: channels of the Jaguar I-PACE recording. 67

List of Tables

1.1 Existing solutions and their limits for BMS validation work. . . . 3
1.2 Comparison of Waterfall, Scrum and the Unified Process. . . . 6
1.3 Phases of the Unified Process in the project. . . . 8

2.1 Functional requirements, by theme and priority. . . . 11
2.2 Non functional requirements, classified with FURPS+. . . . 12
2.3 Summary of the use cases. . . . 14
2.4 Use case *Import log*, with *Select vehicle*, *Check model status* and *Detect anomaly* (offline analysis). . . . 15
2.5 Use case *Replay log*. . . . 15
2.6 Use case *Request explanation*. . . . 16
2.7 Use case *Export report*. . . . 16
2.8 Use case *Connect hardware* (live capture). . . . 17
2.9 Initial risk list. . . . 19

3.1 Scoring and alerting rules of the two detection paths. . . . 34
3.2 Dataset files used in the project. . . . 35
3.3 Candidate machine learning algorithms for the anomaly score. . . . 36

4.1 Iterations of the construction phase. . . . 38
4.2 Vehicle profiles with DBC decoding. . . . 41
4.3 Root causes of the 15,945 alerts of the first pipeline. . . . 44
4.4 Before and after the corrections, on the Kia EV6 recording (97,741 frames). . . . 48
4.5 Defects corrected during the second iteration. . . . 49
4.6 Parameters of the final models. . . . 53
4.7 Status of a model and what it allows. . . . 55
4.8 Final models scored on their holdout recordings. . . . 57
4.9 Speed of the live path with the final models, on the first 10,000 frames of each holdout. The model call is given as mean and 99th percentile. . . . 58

5.1 Stages of the validation. . . . 68
5.2 Testing matrix: what was covered, and how. . . . 69
5.3 Requirements traceability, from requirement to implementation to validation. . . . 70
5.4 User guide: main tasks. . . . 72

List of Abbreviations

2TUP Two Track Unified Process.

API Application Programming Interface.

BMS Battery Management System.

CAN Controller Area Network.

CSV Comma-Separated Values.

DBC CAN Database.

ECU Electronic Control Unit.

FURPS+ Functionality, Usability, Reliability, Performance, Supportability, plus constraints.

HTTP Hypertext Transfer Protocol.

LLM Large Language Model.

LOF Local Outlier Factor.

MF4 ASAM Measurement Data Format version 4.

MVVM Model-View-ViewModel.

OBD-II On-Board Diagnostics, second generation.

REST Representational State Transfer.

SVM Support Vector Machine.

UDS Unified Diagnostic Services.

UML Unified Modeling Language.

UP Unified Process.

WPF Windows Presentation Foundation.

General Introduction

A modern vehicle is a small network of computers on wheels. An electric car contains many Electronic Control Units (ECUs) that control braking, battery management, motor torque and driver assistance, and they exchange data mostly over the Controller Area Network (CAN) bus.

The CAN protocol [1] was designed in the 1980s for reliability. Every node can send and receive frames, and when two nodes talk at the same time, the message with the highest priority wins. This works very well on a closed network where every node can be trusted. It was never meant to resist someone who gains access to the bus: a frame carries no field for authentication and no encryption, so the bus cannot tell a genuine frame from a forged one.

Over the last fifteen years, security researchers have shown many times that this is not a theoretical problem. Experimental analyses of production vehicles [2], [3], a remote attack on an unmodified car [4] and public intrusion detection datasets [5], [6] have all shown that access to the CAN bus can change the physical behaviour of a vehicle. At the same time, electric vehicles have added more safety critical messages to the bus, such as battery management and motor control, and more remote entry points, such as telematics, over the air updates and mobile applications.

Detecting anomalies on a CAN bus is not the same problem as on a computer network: the frames carry at most 8 bytes, most are periodic, and their meaning changes from one vehicle to another. This calls for a system that looks at timing, payload and a learned model of normal behaviour, and that adapts to the vehicle it watches.

This report presents the design, the implementation and the validation of **CANvisionNative**, a diagnostic and validation platform for the CAN traffic of Battery Management Systems. It gives a BMS engineer one view of the traffic, through replay, monitoring and anomaly detection, explains the anomalies in plain language and exports the results. A large part of the project went into validation: six defects were found and corrected on a real vehicle recording, and the rebuilt models were evaluated on recordings they had never seen.

This report follows the phases of the Unified Process. Chapter 1 presents the general framework of the project: the host company, the context, the existing solutions, the problem statement, the proposed solution, its objectives and the development methodology. Chapter 2 presents the inception phase: the actors, the functional and non functional requirements, the use case model, the initial risks and the plan. Chapter 3 presents the elaboration phase: the technical choices, the architecture, the analysis model, the dynamic view, the detection architecture, the framing of the machine learning problem and the architecture baseline. Chapter 4 presents the four iterations of the construction phase: the first detection pipeline and client, its validation and correction on a

Kia EV6 recording, the hardware layer and its bench rig, and the rebuild of the models with their evaluation on recordings kept out of training. Chapter 5 presents the transition phase: the final release, its tests, the traceability of the requirements, the deployment and documentation, and the scope within which the software can be considered stable. The report ends with a general conclusion.

Chapter 1

General Framework

1.1 Introduction

This chapter sets the frame of the project: the host company and team, the context of the work, the existing solutions, the problems the project addresses, the solution we propose, its objectives, and the development process we followed, the Unified Process.

1.2 Host organization

1.2.1 Presentation

This graduation project was carried out within a work and study program (alternance) at **Capgemini Engineering** (Figure 1.1), the engineering and research and development branch of the Capgemini group. The group, headquartered in France, provides consulting, digital transformation and engineering services to many industries. The automotive sector is one of them, where Capgemini works both on software systems and on embedded engineering.



Figure 1.1: Logo of Capgemini Engineering.

1.2.2 Department and work context

The program takes place in the Automotive division of Capgemini Tunisia, within the Battery Management Systems (BMS) team, which develops and validates the embedded software of battery management functions for electric and hybrid vehicles. Since September 2024, it has combined a full time position in this team, five days a week from 9 am to 6 pm, as a quality assurance (QA) tester of battery management software for BMW Group, with classes at ESPRIT on Saturdays. A large part of this work consists in checking the messages that the ECUs exchange on the CAN bus: whether a cell voltage, a temperature or a state of charge is sent at the expected rate and with a plausible value.

CANvisionNative is not one of the team's deliverables: it was built alongside this position, from February 2026. The idea came from the difficulties met in this daily validation work. Working inside a team that knows ECU communication and automotive embedded software helped keep the project close to real needs.

1.3 Project context

An electric vehicle relies on its CAN bus to let its ECUs exchange data, and the Battery Management System is one of the busiest of them: it sends cell voltages, temperatures, currents and the state of charge many times per second. Validating a BMS therefore means reading a large amount of CAN traffic and deciding, frame after frame, whether it behaves as expected.

The CAN protocol was designed for reliability on a closed network. It gives no sender identity and no authentication, and the security literature has shown that it can be abused through message injection, flooding or replay [2], [7]. A validation toolchain that does not watch the bus cannot notice a malicious frame any more than a faulty one.

In this context, an engineer needs to work on recorded logs as well as on a live bus, to replay a session after a change, to see quickly which frames depart from the usual behaviour of the vehicle, and to understand why.

1.4 Study of existing solutions

The existing solutions fall into three categories. Figure 1.2 shows the commercial and open source tools and the diagnostic standards discussed below.



CANoe, CANalyzer SavvyCAN candump (can-utils) SAE J1979 (OBD-II) ISO 14229 (UDS)

Figure 1.2: Existing tools and diagnostic standards: Vector CANoe and CANalyzer, SavvyCAN, candump from the Linux can-utils, and the SAE J1979 and ISO 14229 standards used by diagnostic scanners.

1.4.1 CAN analysers

Commercial tools such as Vector CANoe [8] and CANalyzer are the professional standard for observing CAN traffic: they capture raw frames, decode them with a CAN Database (DBC) file and replay a recorded log. They show what is on the bus, but not whether a frame is normal for this vehicle, unless custom scripts are written on top of the product.

1.4.2 Intrusion detection research

Research on CAN intrusion detection [7] follows three families. **Signature based detection** compares the traffic with known attack patterns and cannot recognise a new one. **Specification based detection** writes the expected behaviour by hand for every vehicle. **Anomaly based detection** learns normal traffic, for example from the timing of the messages [9], [10] or from its entropy [11], and flags what deviates from it. These detectors usually rely on one signal, aim at attacks rather than at explaining the bus, and are evaluated on public benchmarks [5], [6].

1.4.3 Viewers and diagnostic scanners

Open source utilities such as candump [12] and SavvyCAN [13] display raw frames, with little control over replay and no notion of what is abnormal. Diagnostic scanners based on On-Board Diagnostics, second generation (OBD-II), defined in SAE J1979 [14], or on Unified Diagnostic Services (UDS), defined in ISO 14229 [15], ask specific ECUs for known parameters, so they only see what they ask for.

1.4.4 Synthesis

Table 1.1 summarises these categories.

Table 1.1: Existing solutions and their limits for BMS validation work.

Solution or category	**Main purpose**	**Strengths**	**Limitations relevant to this project**
CAN analysers (Vector CANoe, CANalyzer)	Observe, decode and replay CAN traffic	Professional standard; capture, DBC decoding and replay through a rich, configurable interface	No judgement on whether a frame is normal for the vehicle without custom scripts
Intrusion detection research	Detect attacks on the CAN bus	Signature based: precise on known attacks; specification based: needs no attack data; anomaly based: learns normal traffic, widely applicable	Usually one signal or one set of rules; aims at attacks rather than at explaining the bus; a model trained on one vehicle rarely transfers; no explanation of an alert
Open source viewers (candump, SavvyCAN)	Display and log raw frames	Lightweight and open source	Little control over replay; no notion of what is abnormal
Diagnostic scanners (OBD-II, UDS)	Request known parameters, such as fault codes, from specific ECUs	Standard request and response protocols	Only see what they ask for; do not monitor the bus as a whole

Three limits come back. A detector built on a single signal misses a problem that does not disturb that signal. A model trained on one vehicle rarely transfers to another, because message layouts, sending rates and value ranges differ, and choosing the right model is often treated as a detail. And no tool turns an anomaly score into an explanation an engineer can act on within one workflow.

1.5 Problem statement

Four problems of day to day BMS validation work motivated the project:

1. **No live view of the traffic:** Reading raw frames says what happened, not whether it should have happened. By the time something odd is noticed in a spreadsheet or a log viewer, the context that would explain it is often lost.
2. **Weak replay:** Replaying a session is central to validation, but the usual tools either replay frames without judging them or accept only a few log formats, so engineers convert files by hand or change tools in the middle of an investigation.
3. **Scattered tools:** No single tool available to the team combines decoding, replay, live capture and anomaly detection. An engineer combines a CAN analyser, a diagnostic scanner, and a spreadsheet for comparisons across sessions.
4. **Alerts without explanation:** When a tool flags something, it is usually a bare number that does not say which pattern caused it or what to check next.

The lack of security of the CAN protocol (ISO 11898 [1]) makes these problems worse. Security is therefore a real but secondary motivation: the first goal is one clear and explainable view of the bus. The question this project answers is therefore: how can we give a BMS engineer one view of the CAN traffic, adapted to each vehicle, that explains its own alerts, and show that it works on real recordings?

1.6 Proposed solution

We propose **CANvisionNative**, a diagnostic and validation platform for the CAN traffic of Battery Management Systems. Diagnostics is its main purpose; live monitoring, log replay and anomaly detection are the means to serve it. It is made of four parts:

- A Windows desktop client (Windows Presentation Foundation (WPF), Model-View-ViewModel (MVVM)) where the analyst imports a log, replays it and reviews the results.
- A Python backend (FastAPI) that reads recordings in several formats, decodes the frames when a DBC file exists, scores each frame and explains the flagged ones in plain language.
- Machine learning models trained separately for each vehicle.

- Three traffic sources: recorded logs, a live simulator, and live capture from a CAN adapter.

CANvisionNative does not compete with detection research; it gathers in one diagnostic workflow what the tools of Section 1.4 offer separately: replay of several log formats, monitoring, anomaly detection per vehicle, and explanations in plain language. The model used for a vehicle is chosen explicitly and its status is shown before use. The platform can be used after the fact, on a recorded log, or as a live monitor when an adapter is connected.

1.7 Objectives

1.7.1 General objective

The general objective is to design, build and validate a diagnostic platform for Battery Management Systems that gives an engineer a unified view of the CAN traffic, adapts to the vehicle being analysed, and explains its own alerts.

1.7.2 Specific objectives

The specific objectives are:

1. Build a detection pipeline that combines a machine learning model with timing, payload and persistence checks.
2. Accept the common recording formats (Comma-Separated Values (CSV), ASAM Measurement Data Format version 4 (MF4), candump, Vector and PCAN logs), a live simulator and live capture through python-can.
3. Train a separate model for each vehicle, and put into production only the models evaluated on a recording kept out of their training.
4. Build a desktop client to import a log, replay it, review the anomalies and export a report.
5. Add an explanation layer, separate from detection, using rules and a Large Language Model (LLM).
6. Validate the pipeline on real recordings with reproducible evidence, and correct every defect found.

1.8 Development methodology

1.8.1 Comparison: Waterfall, Scrum and the Unified Process

Before organising the work, we compared three processes usually considered for a project of this size: the Waterfall model [16], Scrum [17], and the Unified Process (UP) [18], with its variant Two Track Unified Process (2TUP) [19], which separates a functional branch and a technical branch before joining them in the design.

We chose the UP. Two main uncertainties of the project could not be estimated at the start: how well a model trained for one vehicle would perform, and how many defects a careful validation would uncover. A Waterfall plan would have frozen the detection design and the validation scope before either was known. Scrum assumes a team with separate roles and regular ceremonies, which one developer cannot reproduce. The UP keeps what fits a project of one person: iterations that each deliver a working increment, use cases that drive the work, an architecture tested early, and a list of risks that decides the order of the work.

Table 1.2 compares the three processes on the criteria that mattered for this project.

Table 1.2: Comparison of Waterfall, Scrum and the Unified Process.

Criterion	**Waterfall**	**Scrum**	**Unified Process (2TUP)**
Organisation	Sequential phases, each finished before the next	Short sprints with a prioritised backlog	Four phases, each made of iterations
Requirements	Frozen at the start	Change freely from sprint to sprint	Captured as use cases, refined at each iteration
Architecture	Designed once, early	Emerges from the increments	Central: an executable architecture is built and tested early
Risks	Found late, at integration or test	Handled implicitly, sprint by sprint	Listed at the start; the riskiest parts are done first
Documentation and models	Heavy, produced at each phase	Light	UML models as the main artefacts
Team needed	Any	A team with separate roles and regular ceremonies	Scales from one developer to a team
Fit for this project	Poor: the model quality and the number of defects were unknown	Partial: one developer cannot run the ceremonies	Good: iterative, driven by use cases and risks, centred on the architecture

1.8.2 Principles and phases of the Unified Process

The UP [18], [20] rests on four principles:

- **Driven by the use cases:** The use cases capture what the users need, and every iteration implements and tests some of them.
- **Centred on the architecture:** An executable architecture is built early and the rest of the system grows on it.
- **Iterative and incremental:** The work is cut into iterations, each one delivering a tested increment.
- **Driven by the risks:** The main risks are listed at the start, and the iterations are ordered so that the riskiest parts are tackled first.

The life of a project is divided into four phases, each closed by a milestone. **Inception** defines the vision, the scope, the main use cases and the risks. **Elaboration** analyses the requirements and builds the architecture baseline. **Construction** builds the system in iterations. **Transition** delivers the system to its users: final tests, deployment and documentation. In every phase, the work spans the same disciplines, in varying proportions: requirements, analysis and design, implementation, test and deployment.

1.8.3 The Unified Process applied to CANvisionNative

Table 1.3 and Figure 1.3 show how the project followed these phases. The roadmap of the project listed eight milestones, M1 to M8: the live foundation, the hardware layer, the training of the vehicle models, the explanation engine, the real time charts, session recording, integration, and the decoding pipeline. The live foundation, M1, built the executable architecture of the elaboration phase. The construction phase was made of four iterations: the first delivered milestones M3 to M8; the second validated this first pipeline on a real recording and corrected its defects; the third delivered milestone M2, the hardware layer; and the fourth rebuilt the models and evaluated them on recordings they had never seen.

Table 1.3: Phases of the Unified Process in the project.

Phase	**Period (2026)**	**Content and deliverables**	**Presented in**
Inception	February	Problem, existing tools, vision, use cases, initial risks and plan	Chapters 1 and 2
Elaboration	February to March	Analysis and design models; executable architecture: client and backend, one runtime engine for replay, simulation and live capture (M1)	Chapter 3
Construction, iteration 1	March to June	First detection pipeline and client: M3 to M8	Section 4.3
Construction, iteration 2	June to July	Validation and correction of six defects on a Kia EV6 recording	Section 4.4
Construction, iteration 3	August to September	Hardware layer and bench rig (M2)	Section 4.5
Construction, iteration 4	September	Model rebuild and evaluation on recordings kept out of training	Section 4.6
Transition	September to October	Final release, tests, deployment guide and report	Chapter 5

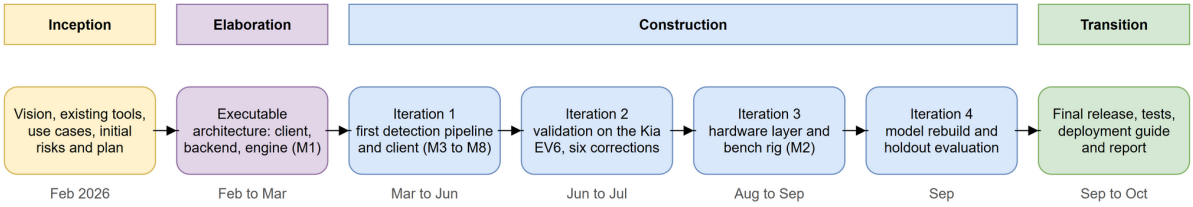


Figure 1.3: Phases and iterations of the Unified Process in the project.

Two rules applied to every iteration. A claim about the system was accepted only once shown on real data, and a defect was considered fixed only once the same test, run again, gave the expected result. As the only developer, we did not reproduce the roles of a team; each iteration ended with a review of its increment against its use cases before the next one started.

1.8.4 Modelling language and tools

The models of this report follow the Unified Modeling Language (UML) [21]. The following diagrams are used:

- **Use case diagram:** It shows the actors of the system and the services the system offers them.
- **Activity diagram:** It shows the flow of actions of a use case, with its decisions, divided between the actors and the system.

- **Class diagram:** It shows the classes of the software, with their attributes, their operations and the relations between them. The domain model, a class diagram of the concepts of the problem without software details, and the data model, which describes the tables of the database, are drawn in the same notation.
- **Component diagram:** It shows the parts of the system and the interfaces through which they depend on each other.
- **Deployment diagram:** It shows the machines and the processes on which the components run, and how they communicate.
- **Sequence diagram:** It shows, for one scenario, the numbered messages exchanged over time between an actor and the boundary, control and entity objects of the system.
- **State diagram:** It shows the states an object goes through and the events that move it from one state to another.

The diagrams were drawn with draw.io from a script that keeps every diagram in a file that can be edited. The source code was versioned with Git and GitHub, and the report was typeset with LaTeX.

1.9 Conclusion

This chapter presented the host company, the context of the project, the existing solutions and their limits, the problem and the solution we propose, its objectives, and the Unified Process that organised the work. The next chapter presents the inception phase: the capture of the requirements.

Chapter 2

Inception: Requirements Capture

2.1 Introduction

The inception phase turns the objectives of Chapter 1 into requirements. This chapter identifies the actors, states the functional and non functional requirements, builds the use case model that drives the iterations, lists the initial risks, and plans the work by phase and iteration.

2.2 Actors

An actor is a role played outside the system that interacts with it. CANvisionNative has three:

- **Analyst (main actor).** A BMS engineer or tester who imports logs, starts replays and live sessions, reviews the alerts, asks for explanations, exports reports and configures the platform.
- **Vehicle or CAN bus (secondary actor).** An external system that sends frames on the bus; it is the source of the traffic in a live session, through a CAN adapter.
- **LLM provider (secondary actor).** An external service that writes the natural language text of an explanation: Google Gemini in the cloud, or a local Ollama model.

The backend of CANvisionNative is not an actor: it is part of the system under design, together with the desktop client.

2.3 Functional requirements

Table 2.1 groups the functional requirements by theme and gives each one a priority: *must* for the requirements without which the platform has no purpose, *should* for those expected in the final release, and *could* for those that are useful but not essential.

Table 2.1: Functional requirements, by theme and priority.

ID	Requirement	Priority
Input and vehicle		
FR1	Import a recorded log in CSV, MF4, Vector (.asc), candump (.log) or PCAN text (.txt) format, and convert it into one common frame stream made of a timestamp, an identifier and a payload.	Must
FR2	Let the analyst choose the vehicle that produced the log, so that the matching trained model scores the frames and the matching DBC file, when one exists, decodes them.	Must
FR12	For a loaded log, show which signals it carries (the decoded signals when a DBC file exists, otherwise the most active raw payload bytes, marked as not decoded) and describe the dataset: its size, its duration, and whether the selected model was trained or tested on it.	Should
FR13	Show, for every vehicle model, whether it has been evaluated on a recording kept out of its training, and refuse an analysis with a model whose files cannot be shown to belong to one training run.	Must
Analysis		
FR3	Replay the imported frames in order, at a speed chosen by the analyst (0.5, 1, 2, 4 or 8 times real time, or 32 times as the maximum), and show the progress of the replay.	Must
FR4	Score every frame for which a model exists and flag the frames whose score passes the calibrated threshold of that model; report the frames that cannot be scored as unresolved, with the reason, instead of counting them as normal; complement the model score with timing, payload and persistence checks.	Must
FR5	Give each alert a severity level derived from its score.	Must
Explanation and reporting		
FR6	Explain any alert on request, in natural language, with the dominant signal, the likely cause and a recommended action.	Must
FR7	Export the alert list as a CSV file and as a formatted PDF report.	Must
Live operation		
FR8	Generate a simulated live CAN stream, including a simulated anomaly burst, to test the detection pipeline without a vehicle.	Could
FR9	Connect to a physical CAN adapter supported by python-can (for example a serial adapter using the slcan protocol, a PEAK PCAN adapter or SocketCAN) and send the received frames into the same runtime engine as replay; offer the live session only when a USB device is detected.	Should
FR10	Record a live or simulated session to a log file, optionally with an injected and labelled anomaly, for later replay or training.	Could
Configuration		
FR11	Let the analyst set the key of the LLM provider, the hardware settings and an alert threshold, and keep these settings between sessions.	Should

2.4 Non functional requirements

Table 2.2 classifies the non functional requirements with the Functionality, Usability, Reliability, Performance, Supportability, plus constraints (FURPS+) model.

Table 2.2: Non functional requirements, classified with FURPS+.

Category	**ID**	**Requirement**
Usability	NFR3	An analyst without a background in machine learning shall be able to import a log, read the alert list and understand the explanation of an alert without external documentation.
Reliability	NFR2	A detection defect shall be shown and corrected with concrete evidence, never by tuning a threshold until the numbers look better.
	NFR6	If the LLM provider cannot be reached, the system shall still produce a rule based explanation instead of failing.
Performance	NFR1	An offline analysis of a recording of several hundred thousand frames shall finish in a few minutes on a standard desktop computer; live scoring shall keep up with the bus in real time.
Supportability	NFR4	Every supported log format shall enter the detection pipeline through the same frame representation, so that the results do not depend on the format used to record the data.
	NFR5	Adding a vehicle shall only require a DBC file and a trained model folder that passes the promotion checks, without any change to the detection engine.
Constraints (+)	NFR7	The desktop client shall run on Windows 10 or 11 with .NET Framework 4.8 and WPF; the backend shall run as a local Python process (3.11 or later) that exposes a REST API on the same machine; only live capture needs extra hardware, a CAN adapter supported by python-can.

2.5 Use case model

2.5.1 Global use case diagram

Figure 2.1 shows the use case diagram. Replaying a log, running a live simulation and connecting the hardware include *Detect anomaly*, because the three input modes feed the same runtime engine. Selecting a vehicle includes *Check model status*, since a model is only used once its status allows it. *Request explanation* is an optional extension of *Review alerts*, and the only use case that involves the LLM provider.

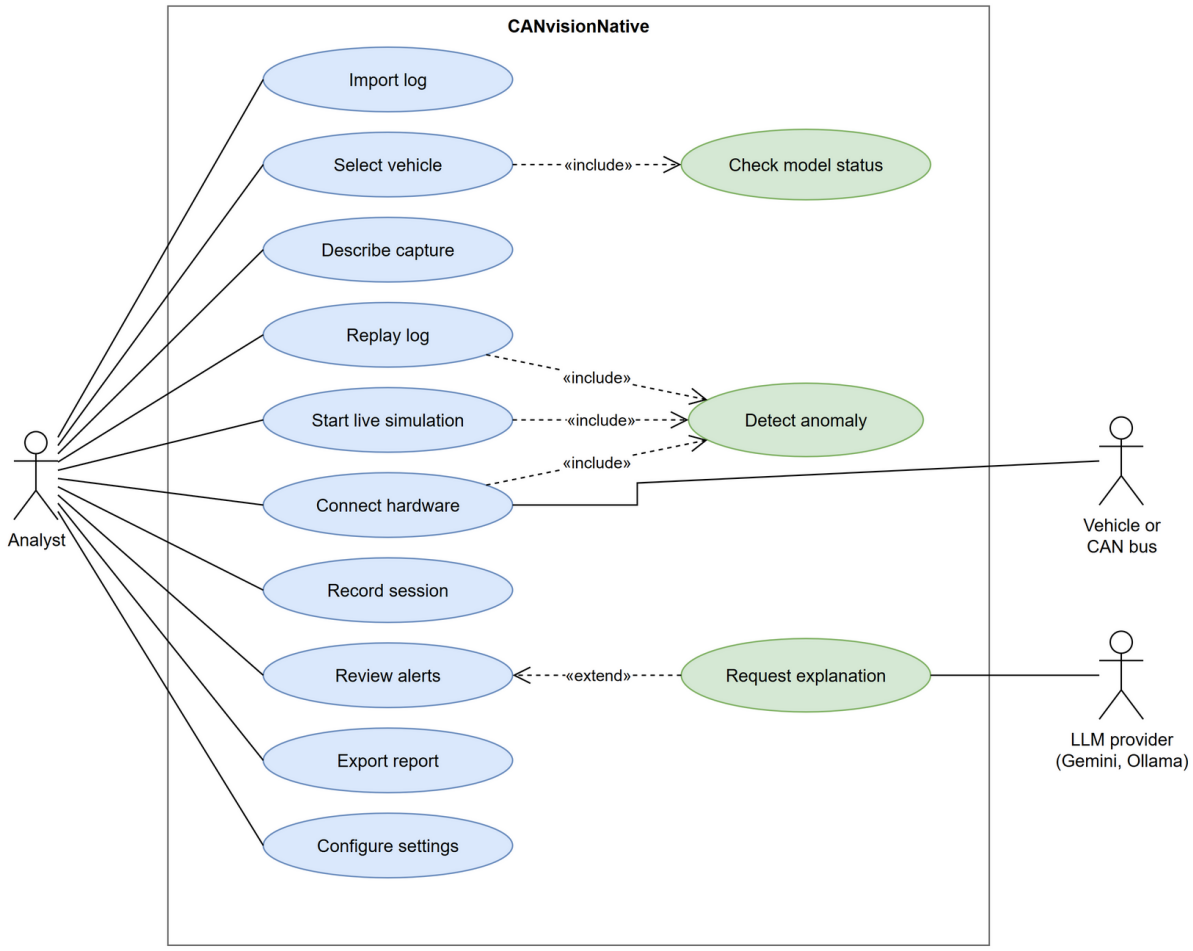


Figure 2.1: Use case diagram of CANvisionNative.

Table 2.3 summarises the use cases and the requirements they cover.

Table 2.3: Summary of the use cases.

Use case	**Description**	**Req.**
Import log	The analyst selects a log file; the system recognises its format and converts it into the common frame stream.	FR1
Select vehicle	The analyst chooses the vehicle model used to score the log and the DBC file used to decode it.	FR2
Check model status	The system tells whether the chosen model may be used offline or live, and refuses it otherwise.	FR13
Describe capture	The system shows the signals of the capture and a plain description of the dataset.	FR12
Replay log	The system replays the frame stream at the chosen speed and runs *Detect anomaly* on it.	FR3
Start live simulation	The system generates a simulated stream, with an optional anomaly burst, and runs *Detect anomaly* on it.	FR8
Connect hardware	The system reads live frames from a CAN adapter and runs *Detect anomaly* on them.	FR9
Detect anomaly	The system scores each frame, reports the frames it cannot score, and raises alerts with a severity.	FR4, FR5
Record session	The system records a live or simulated session to a log file.	FR10
Review alerts	The analyst browses the alert list and the signals behind a selected alert.	FR4, FR5
Request explanation	The system explains a selected alert in plain language.	FR6
Export report	The analyst exports the alert list as a CSV file or a PDF report.	FR7
Configure settings	The analyst sets the LLM key, the hardware parameters and the alert threshold.	FR11

2.5.2 Textual descriptions

Tables 2.4 to 2.8 describe the five main use cases in detail: their preconditions, nominal scenario, alternative scenarios and postconditions.

Table 2.4 describes the offline analysis of a log: the use case *Import log*, with the use cases *Select vehicle*, *Check model status* and *Detect anomaly* that it includes.

Table 2.4: Use case *Import log*, with *Select vehicle*, *Check model status* and *Detect anomaly* (offline analysis).

Actor	Analyst.
Preconditions	The backend is running; a log file in a supported format is available.
Nominal scenario	1. The analyst chooses to import a log on the Home screen and selects a file. 2. The system recognises the format and reads the frames into the common frame stream. 3. The system lists the vehicle models with their status. 4. The analyst selects a model and starts the analysis. 5. The system checks that the model may be used offline. 6. The system scores every frame with the model of its identifier, counts the unresolved frames with their reasons, and merges bursts of alerts. 7. The system shows the results on the Dashboard.
Alternative scenarios	2a. The file cannot be read: the system shows the reason and the analysis does not start. 5a. The model may not be used offline (held back, not validated or from the first design): the system refuses the analysis with the reason, and the analyst can choose another model.
Postconditions	The alerts and the counts of the analysis are available to every screen of the client.
Requirements	FR1, FR2, FR4, FR5, FR13.

Table 2.5 describes the use case *Replay log*, which plays an imported log again through the runtime engine.

Table 2.5: Use case *Replay log*.

Actor	Analyst.
Preconditions	A log has been imported and a vehicle model selected.
Nominal scenario	1. The analyst opens Log Playback, chooses a speed and starts the replay. 2. The system starts a replay process that reads the file with the common parser. 3. The system sends every frame to the runtime engine (*Detect anomaly*). 4. The client follows the progress and shows the flagged spans and the alerts on the timeline. 5. At the end of the file, the system publishes the alerts of the replay so that they can be explained.
Alternative scenarios	1a. The analyst pauses, resumes or stops the replay. 3a. A frame that no model can score is counted as unresolved, with its reason.
Postconditions	The replay is completed and its alerts are in the alert list.
Requirements	FR3, FR4, FR5.

Table 2.6 describes the use case *Request explanation*, the only one that involves the LLM provider.

Table 2.6: Use case *Request explanation*.

Actors	Analyst; LLM provider.
Preconditions	An analysis or a replay has produced alerts, which the analyst reviews in Anomaly Intel.
Nominal scenario	1. The analyst selects an alert. 2. The system finds the alert in the current session and builds its context. 3. The system sends the context to Gemini. 4. Gemini returns a text. 5. The system shows the summary, the likely cause and the recommendation, with the name of the engine that wrote them.
Alternative scenarios	2a. The alert does not belong to the current session: the system returns no explanation. 3a. Gemini cannot be reached: the system asks the local Ollama model. 3b. No language model answers: the system writes a rule based text from the measured values.
Postconditions	The explanation is shown; the score, the flag and the severity of the alert are unchanged.
Requirements	FR6, NFR6.

Table 2.7 describes the use case *Export report*, which saves the alert list of a session.

Table 2.7: Use case *Export report*.

Actor	Analyst.
Preconditions	An alert list is shown in Anomaly Intel.
Nominal scenario	1. The analyst chooses to export the list as a PDF report or a CSV file. 2. The system asks for a file name. 3. The system writes the PDF report (session, summary by severity, identifiers with the most alerts, alerts in time order and explanations) or the CSV file (one row per alert).
Alternative scenarios	2a. The analyst cancels: nothing is written.
Postconditions	The report or the file is saved where the analyst chose.
Requirements	FR7.

Table 2.8 describes the use case *Connect hardware*, which starts a live capture from a CAN adapter.

Table 2.8: Use case *Connect hardware* (live capture).

Actors	Analyst; vehicle or CAN bus.
Preconditions	The backend is running and a USB CAN adapter is plugged in.
Nominal scenario	1. The analyst starts a live session on the Home screen. 2. The system opens the hardware setup. 3. The analyst chooses the interface, the port, the bitrate and the vehicle model, and connects. 4. The system builds a live engine for the chosen model and opens the adapter. 5. The system decodes each received frame with the DBC file of the vehicle, when there is one, and scores it (*Detect anomaly*). 6. The live screens show the signals and the alerts. 7. The analyst disconnects and the scoring stops.
Alternative scenarios	1a. No USB device is detected: the live session stays locked. 4a. The model is held back or unknown: the connection is refused with the reason. 5a. No frame arrives for a few seconds: the link is reported as lost.
Postconditions	The adapter is released; the alerts of the session remain available.
Requirements	FR2, FR9.

2.5.3 Activity diagram

Figure 2.2 shows the flow of the main use case, from the import of a log to the display of its results and their replay.

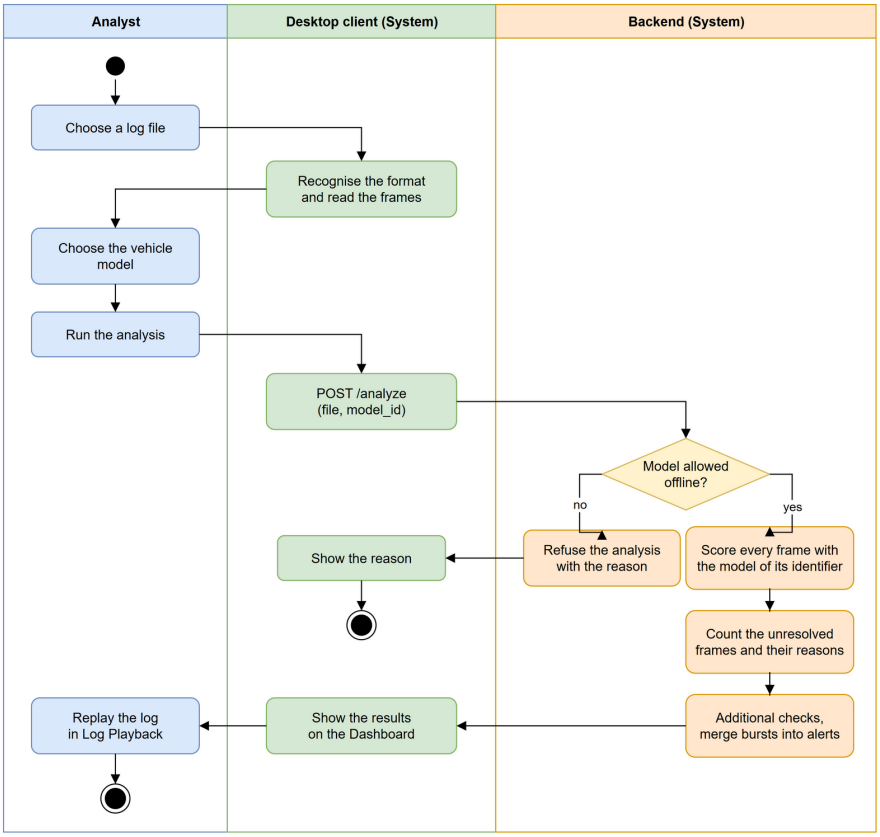


Figure 2.2: Activity diagram: importing, analysing and replaying a log.

2.6 Initial risk list

Table 2.9 lists the risks identified during inception, with their probability, their impact, the planned mitigation and the iteration that addresses each one. Following the UP, the iterations were ordered so that the riskiest parts were tackled early; what actually happened to each risk is discussed in the general conclusion.

Table 2.9: Initial risk list.

Risk	Probability	Impact	Planned mitigation	Iteration
R1. No vehicle available to test live capture	High	Medium	One runtime engine shared by replay, simulation and live capture, so that live capture differs only by its transport; a bench rig to test that transport.	Elaboration, iteration 3
R2. LLM provider unavailable or not authenticated	Medium	Low	A chain of providers: Gemini, then a local Ollama model, then a rule based text.	Iteration 1
R3. Wrong model applied to a vehicle	Medium	High	An explicit choice of the model, checked end to end on a real recording.	Iterations 2 and 4
R4. A model that does not generalise to new recordings	Medium	High	Evaluate every model on a recording kept out of its training before it enters production.	Iteration 4
R5. Live scoring too slow for buses with a high frame rate	Medium	Medium	An offline analysis path for whole recordings; measure the throughput of the live path.	Iteration 4
R6. Too many alerts, so the analyst stops trusting them	Medium	High	Thresholds calibrated on the training data, persistence checks, and unscored frames reported apart instead of counted.	Iterations 2 and 4
R7. Log formats read differently by different paths	Medium	Medium	One common parser and one frame representation for every path.	Iterations 1 and 2
R8. Explanation shown for the wrong alert	Low	High	Tie each explanation to the alert of the current session, and test explanations across datasets of different vehicles.	Iteration 2

2.7 Project plan by phase and iteration

Figure 2.3 shows the plan of the project from February to October 2026, organised by phase of the UP. The whole project was carried out alongside a full time QA position in the BMS team. Inception took the first week of February, while the work on the executable architecture of M1 was starting; the boundaries of the other periods also overlap slightly, because a milestone was being finished while the next one was starting.

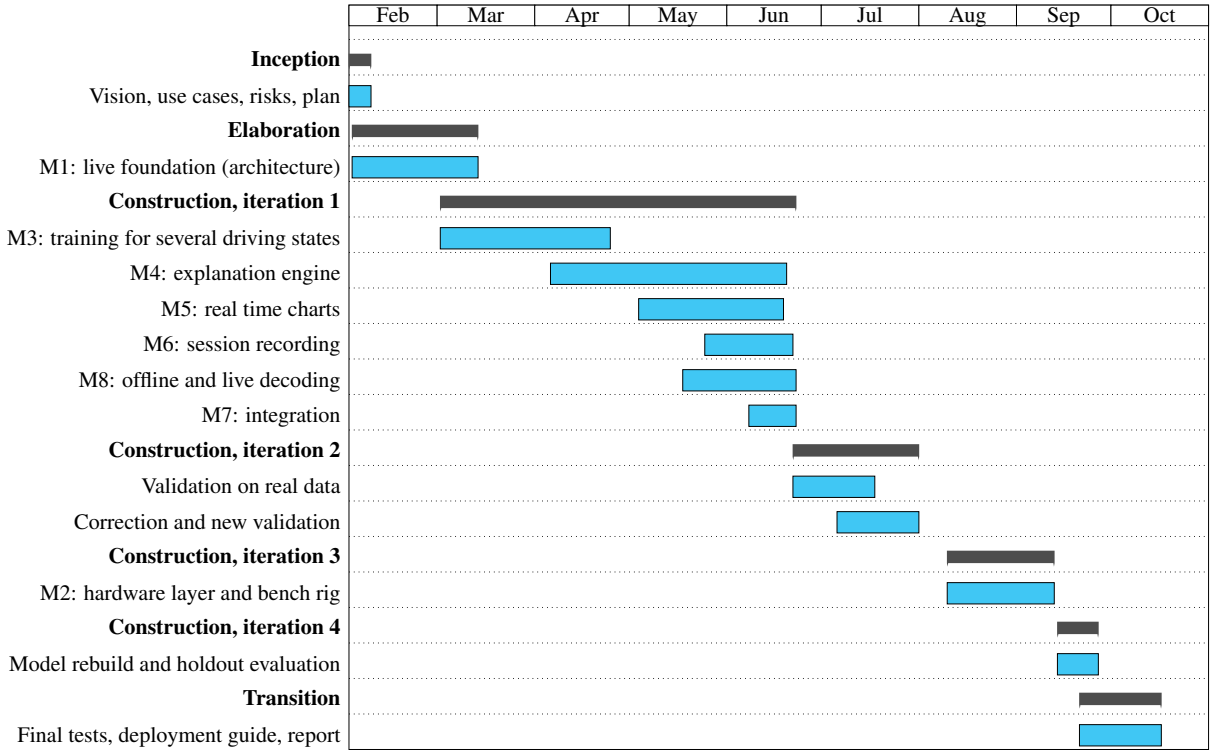


Figure 2.3: Project plan by phase and iteration of the Unified Process, February to October 2026 (Gantt chart).

2.8 Conclusion

The inception phase identified three actors, thirteen functional and seven non functional requirements, the use cases that drive the iterations, and the main risks of the project. The next chapter presents the elaboration phase: the analysis and the architecture.

Chapter 3

Elaboration: Analysis and Architecture

3.1 Introduction

The elaboration phase analyses the requirements of Chapter 2 and builds the architecture on which the construction iterations grow. This chapter says what the system is made of and why: the technical choices, the overall architecture, the analysis model, the dynamic view of the key use cases, the detection architecture, and the framing of the machine learning problem. It ends with the architecture baseline that the phase delivered, and the risks it reduced. How each part was implemented is described in the iterations of Chapter 4.

3.2 Technical choices

Figure 3.1 shows the technologies used in the project.



Figure 3.1: Technologies used in the client, the backend, the machine learning, the tests, the bench rig and the documentation.

The role of each technology is the following:

- **C# and WPF (.NET Framework 4.8):** WPF is the Microsoft framework for native Windows desktop applications, used here with the MVVM pattern and the Community-

Toolkit.Mvvm library. Its data binding suits a client with many screens and charts, and .NET Framework 4.8 was the target platform of the project.

- **Python, FastAPI, pandas and NumPy:** FastAPI [22] builds typed REST APIs that document themselves, and it works directly with the Python data libraries used for the machine learning. The backend is served by uvicorn.
- **scikit-learn:** The scikit-learn library [23] provides the Isolation Forests [24] of the models and the K-Means [25] of the first design. Classical methods covered both needs, so no deep learning framework was required.
- **asammdf, cantools and python-can:** The asammdf library reads ASAM MF4 [26] logs, cantools decodes frames with a DBC file, and python-can [27] gives one interface to many physical CAN adapters, including the Arduino bridge of our bench rig.
- **SQLite:** An embedded database without a server, enough for a system that runs on one workstation.
- **Google Gemini and Ollama:** They are used only by the explanation module: Gemini in the cloud, Ollama as a local model that works without an internet connection. When neither is available, a rule based template writes the explanation.
- **Development environment:** Visual Studio for the client, a Python virtual environment for the backend, PDFsharp to build the PDF report in C#, pytest and xUnit for the automated tests, Git and GitHub to version the project, with the large model files in Git LFS, draw.io for the diagrams and MiKTeX for this report. A Windows workstation served for development, training and validation, and a bench rig of two Arduino boards for live capture.

3.3 Overall architecture

3.3.1 Logical view in layers

CANvisionNative runs as two programs on the same machine, a Windows desktop client and a local Python backend, which talk over Hypertext Transfer Protocol (HTTP), as shown in Figure 3.2 below.

The client only talks to the API layer, which sends each request to the detection engine or to the data layer, and calls the explanation engine only when the analyst asks for an explanation. We chose this separation so that the detection could be tested through the API without the client.

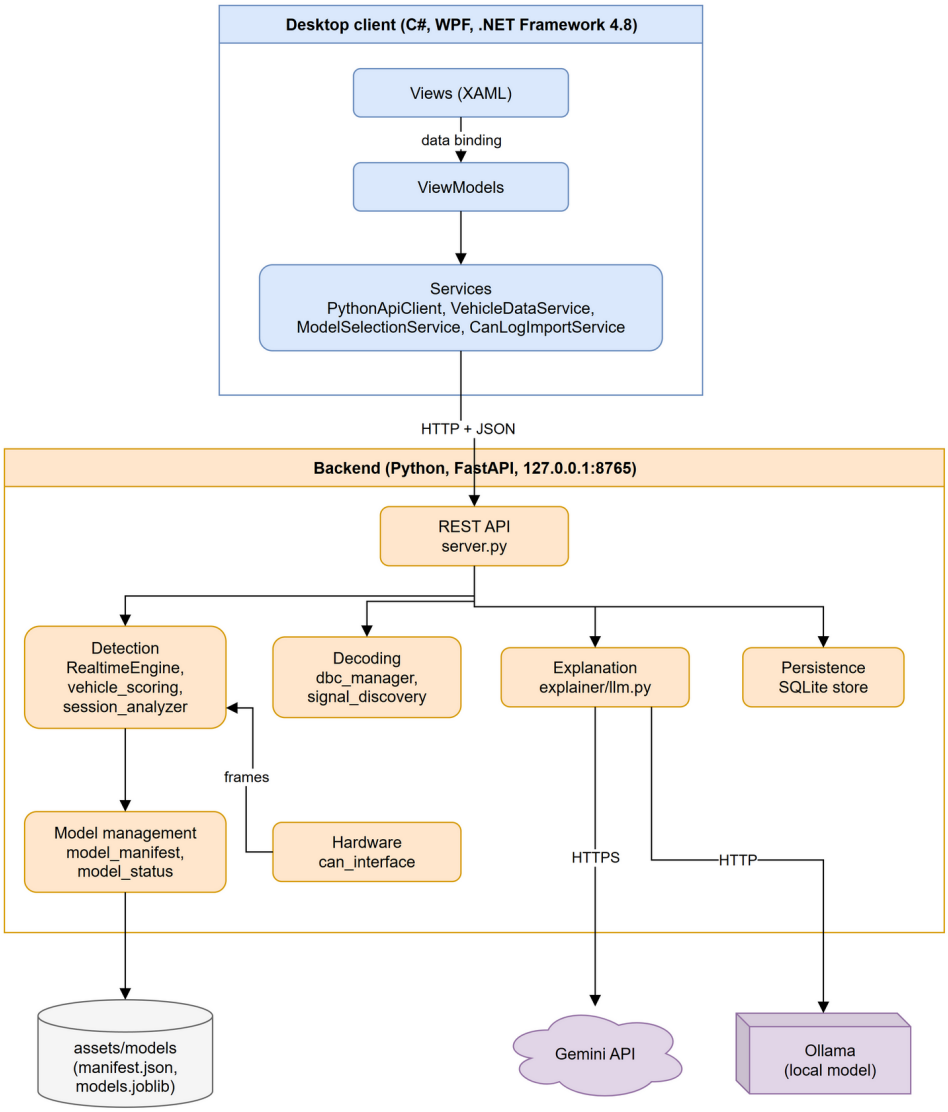


Figure 3.2: Overall layered architecture of CANvisionNative.

3.3.2 Components

Figure 3.3 shows the components and their dependencies. In the client, the views are bound to their ViewModels, which go through a layer of client services to reach the backend. In the backend, the REST API is the only entry point. It starts the offline analyzer for whole recordings, the replay runner and the hardware interface for frame by frame traffic, and calls the explanation engine and the persistence layer. The replay runner and the hardware interface share the runtime engine, and every component that scores frames obtains its models through the model management component, which reads the model files and their manifests. Long tasks, such as a replay or an offline analysis, run in the background while the client follows their status.

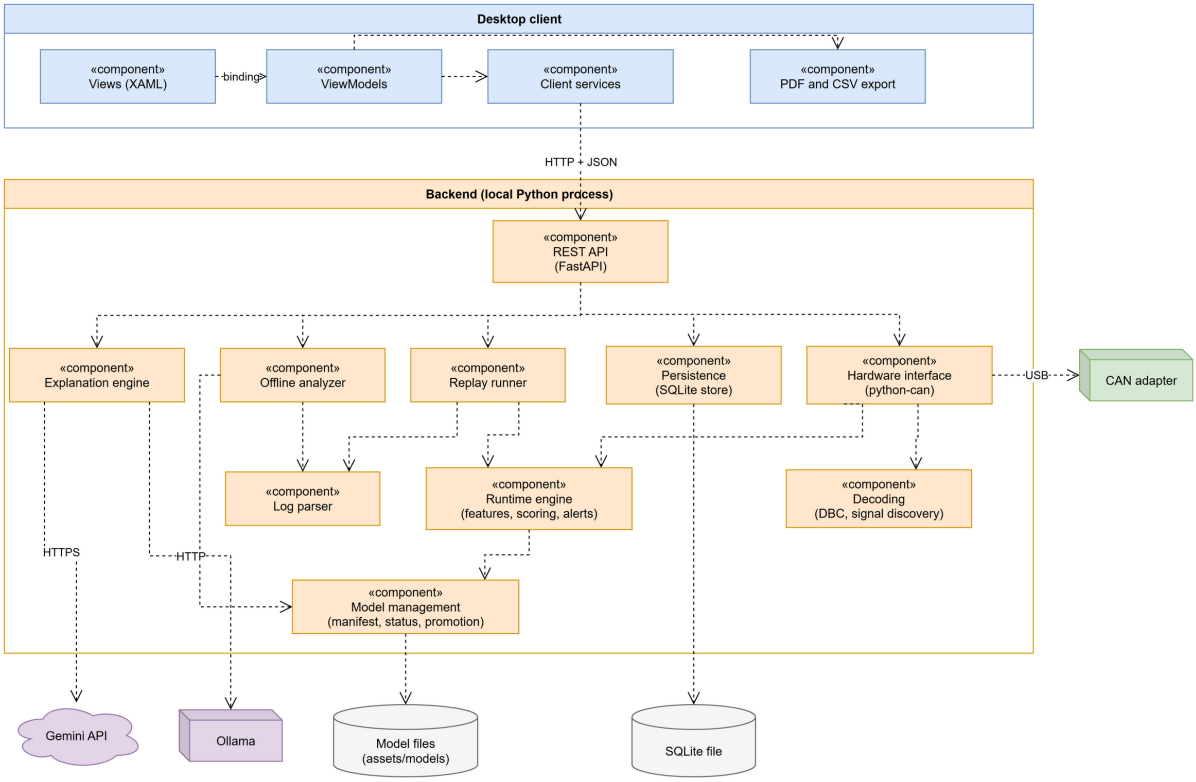


Figure 3.3: Component diagram of the client and the backend.

3.3.3 Client architecture (MVVM)

Each screen of the client (Home, Dashboard, Telemetry, Diagnostics, Log Playback, Anomaly Intel and Settings) has a XAML view and its ViewModel; a Hardware Setup screen opens from Home when a live session starts. Two services sit between the ViewModels and the backend (Figure 3.4): an API client with one method per endpoint, and a data service that keeps the shared state, polls the backend and notifies the screens.

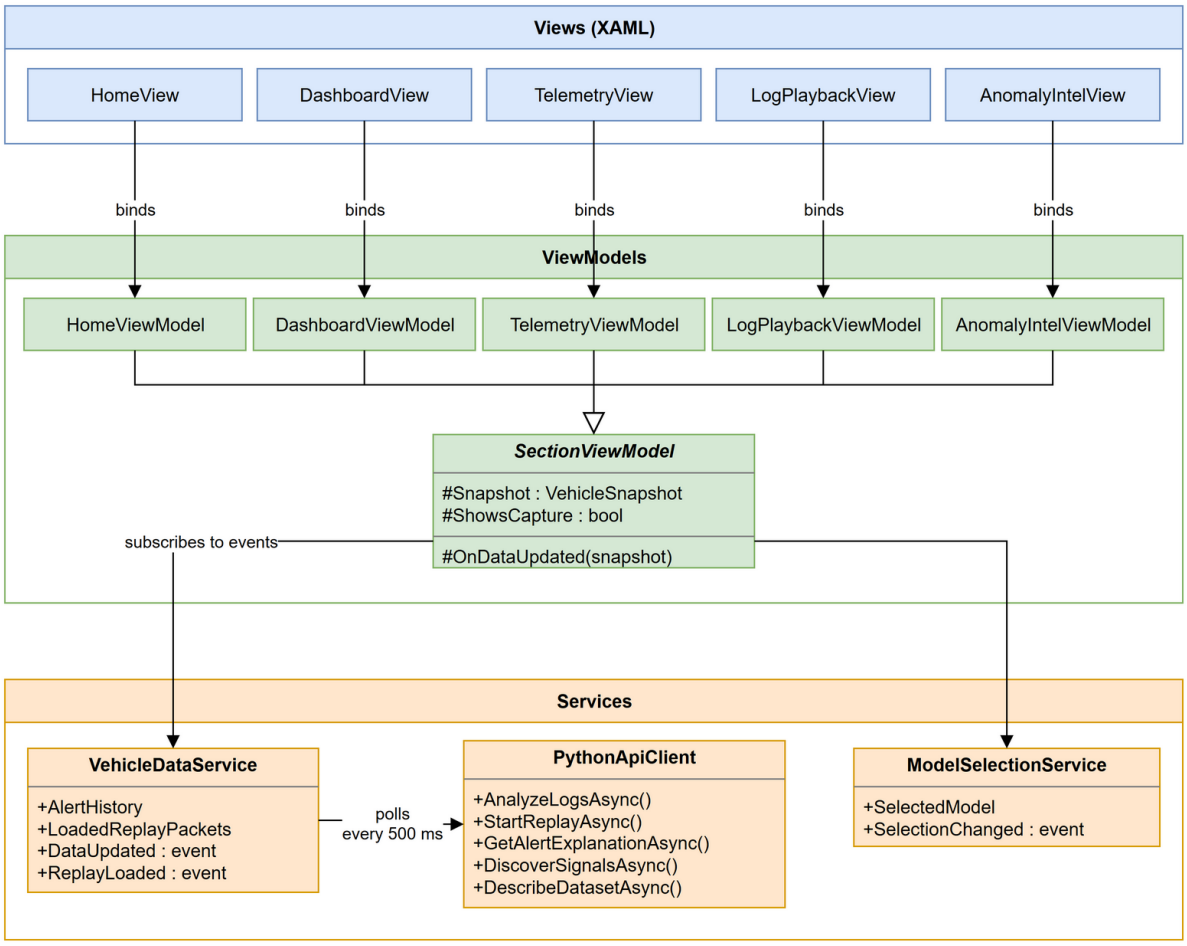


Figure 3.4: Frontend architecture: MVVM views bound to their ViewModels, with two shared services.

3.3.4 Deployment

Figure 3.5 shows the deployment. The client and the backend run on the analyst's workstation, next to the SQLite file, the model folder and an optional local Ollama model; Gemini is reached over HTTPS, and the CAN adapter over USB.

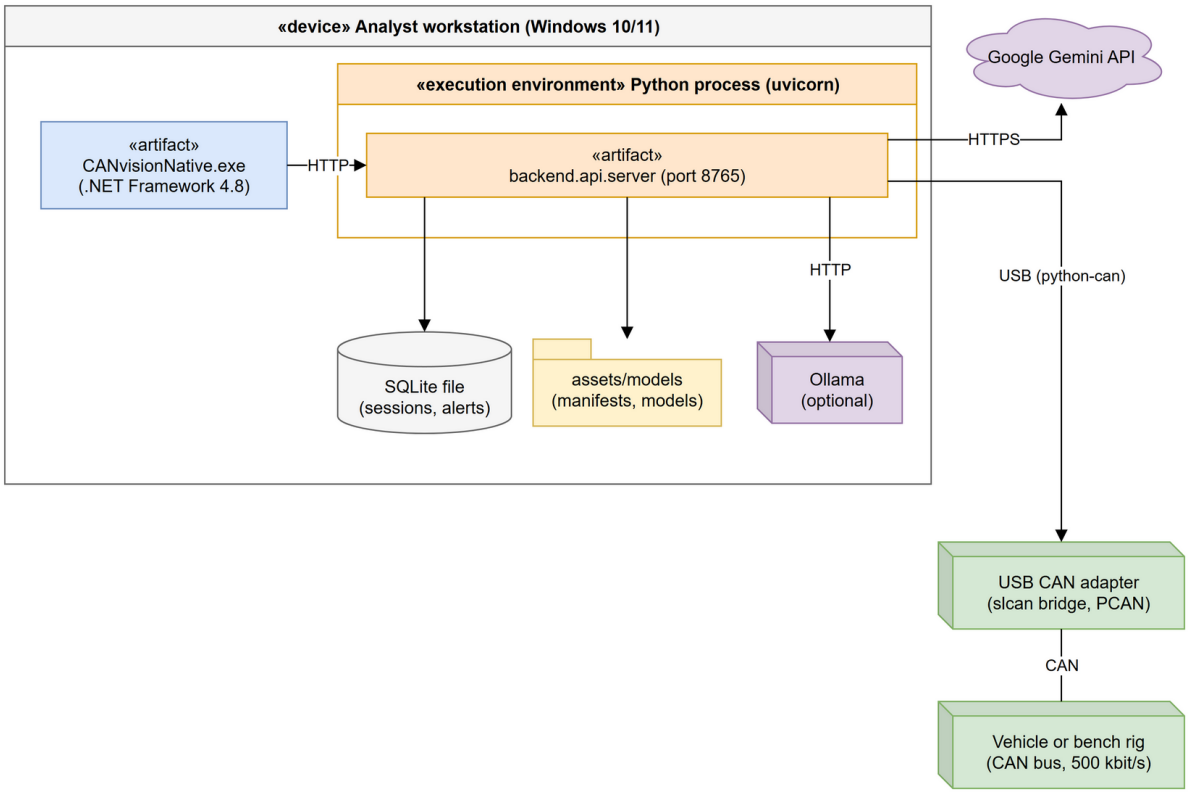


Figure 3.5: Deployment diagram: one workstation, with the external LLM and the hardware links.

3.4 Analysis model

3.4.1 Domain model

Figure 3.6 shows the concepts of the domain. A *vehicle* may have a *profile*, whose DBC files define the *signals* that its frames carry, and a *vehicle model*, made of one *identifier model* per CAN identifier. A *recording* of a vehicle is a sequence of *frames*. A *session*, offline, replay or live, analyses at most one recording with one vehicle model; it counts the frames scored, flagged and unresolved, and produces *alerts*, each of which may receive an *explanation*. A *report* exports one session.

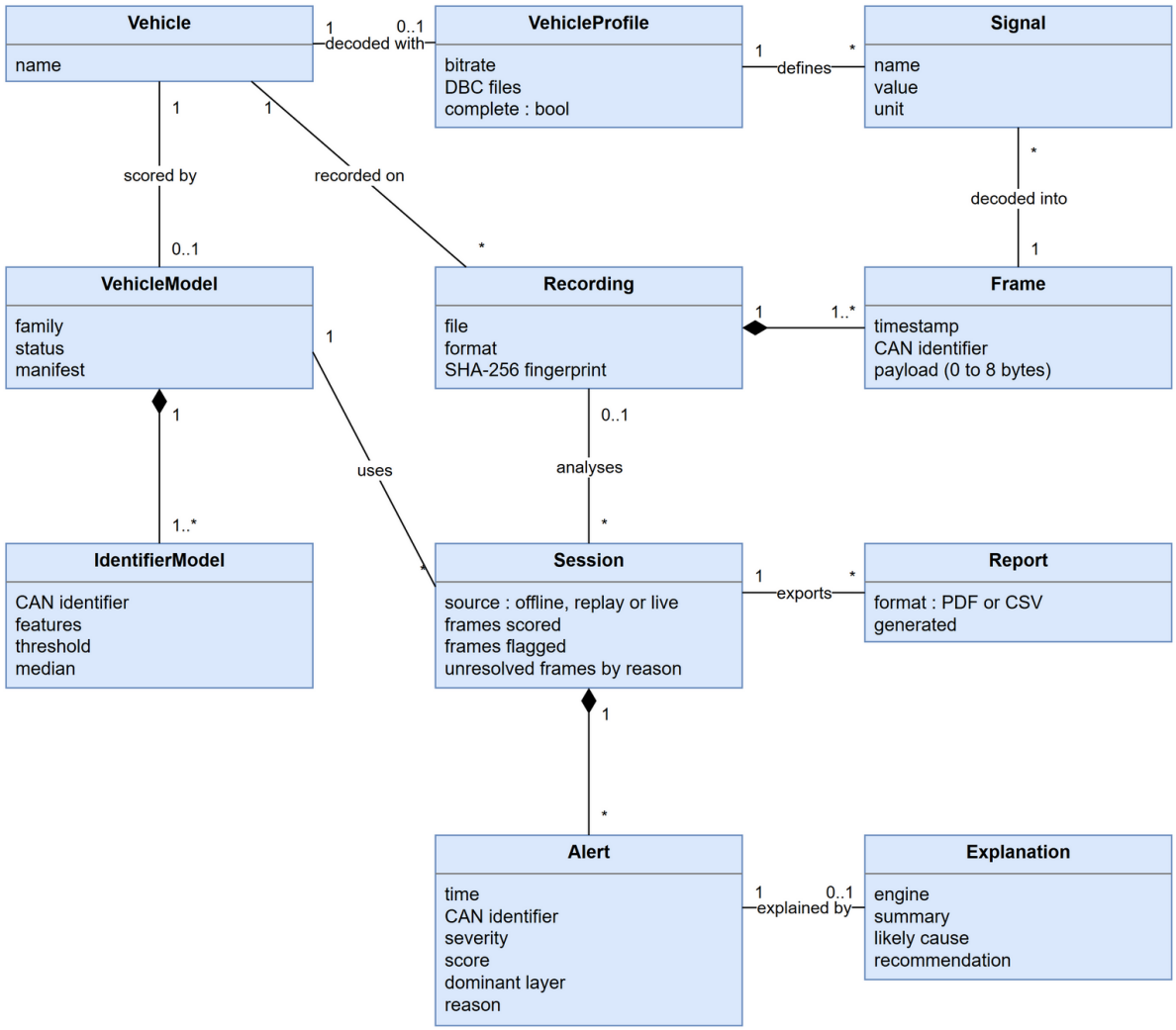


Figure 3.6: Domain model of CANvisionNative.

3.4.2 Class diagrams

The class diagrams refine the domain model into the classes of the software, on the backend and on the client side.

Figure 3.7 shows the classes of the detection core of the backend. RealtimeEngine is the single entry point of replay, simulation and live capture; it asks ScoringEngine to combine the layers and, for a vehicle with a final model, its IdModelFamily, loaded from the manifest, for the score of each identifier. The model status resolver decides which model a vehicle may use on each path, and IDSStatistics counts the frames and alerts of a run.

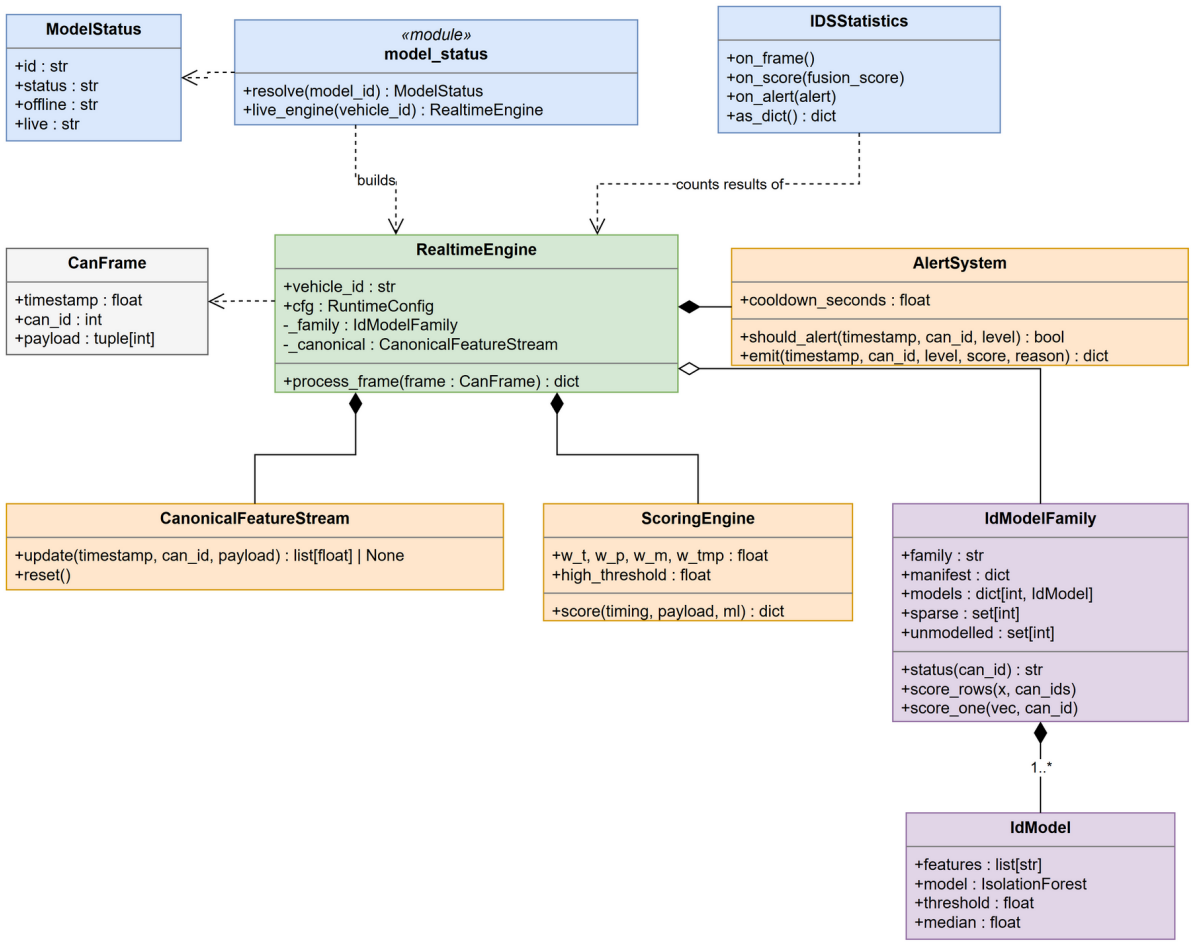


Figure 3.7: Simplified class diagram of the detection core of the backend.

Figure 3.8 details the classes of the client. Every screen ViewModel derives from SectionViewModel, which gives it the shared snapshot of the vehicle and access to VehicleDataService. This service owns the state of the session, the alert history and the global state of the application (AppState), and raises events that the screens follow. It calls the backend through PythonApiClient, and CanLogImportService parses a log before an analysis. ModelSelectionService holds the vehicle model chosen by the analyst, and PdfReportBuilder writes the PDF report from the alert list.

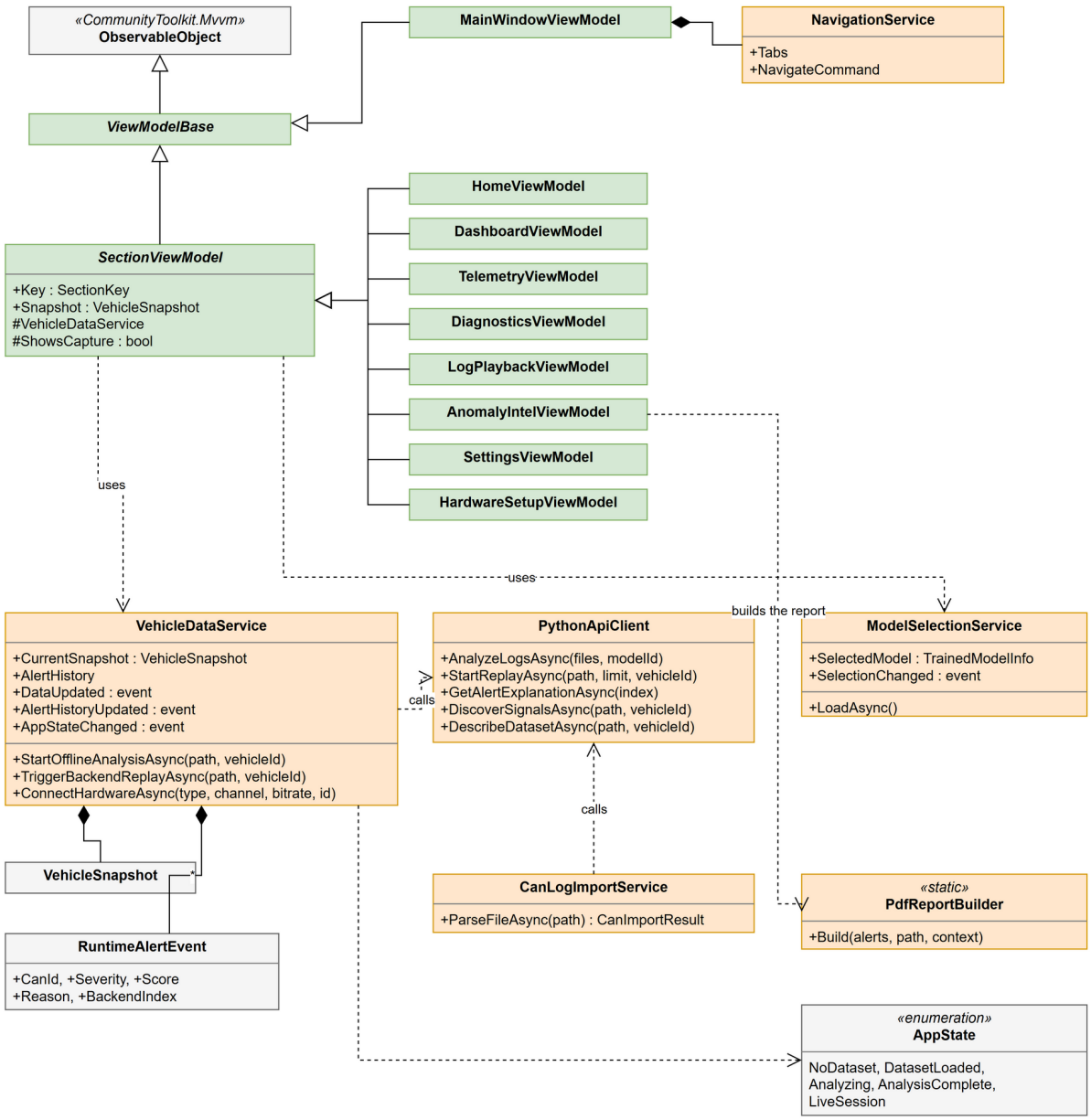


Figure 3.8: Class diagram of the desktop client.

3.4.3 Data model

The backend keeps a small SQLite database (Figure 3.9) so that past sessions can be reviewed from a local file. The `sessions` table records each session and its source; `live_frames` stores the frames of a session with the result of the detection and an optional label; `alert_history` stores the alerts with their severity, score, reason, temporal pattern and recommended response. The vehicle models are not stored in the database but in model folders, each with its manifest.

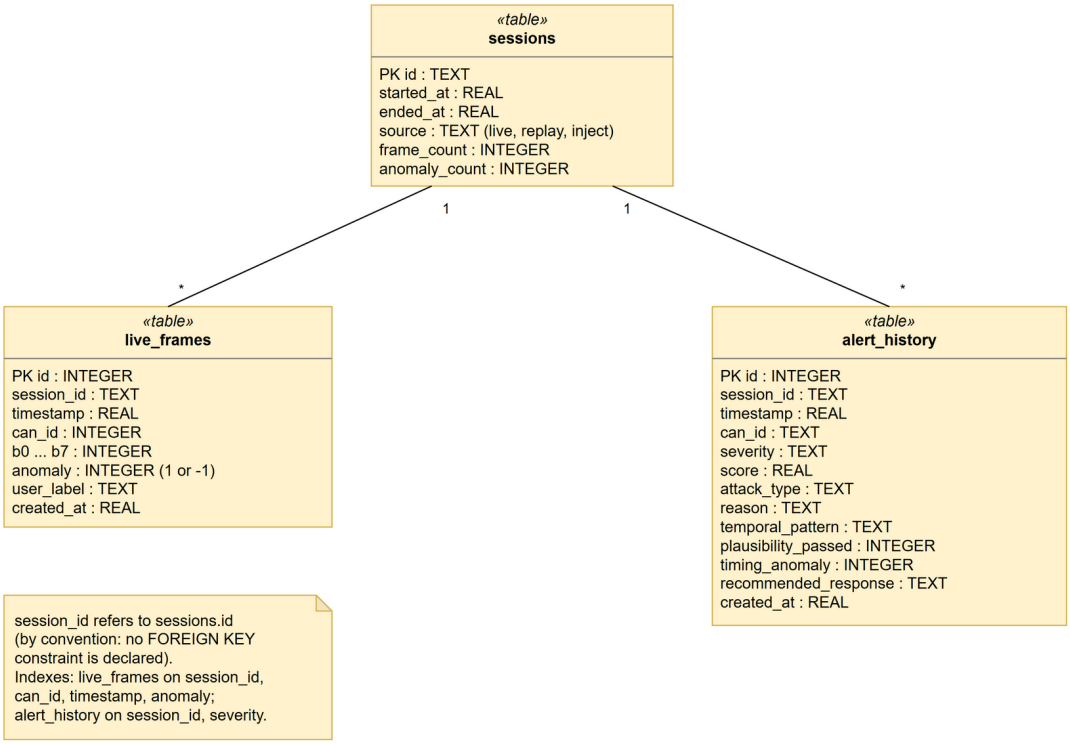


Figure 3.9: Data model of the SQLite store.

3.5 Dynamic view

The two sequence diagrams of this section show the key scenarios of a replay and of an explanation. Each object is marked with its nature: *boundary* for the screen the analyst uses, *control* for the objects that coordinate the scenario, and *entity* for the stored data. The messages are numbered in the order in which they are sent; a synchronous call has a filled arrowhead, an asynchronous message an open arrowhead, and a reply a dashed line.

3.5.1 Life cycle of a replay

A replay runs in a separate process, so that a long file does not block the backend. It reads the file with the same loader as the offline analysis, so that every format becomes the same stream of (timestamp, identifier, payload) frames, and sends each frame to the same runtime engine as the live and simulated paths. The client follows its progress by polling, and the alerts of the replay are published when it ends so that they can be explained. Figure 3.10 shows this life cycle.

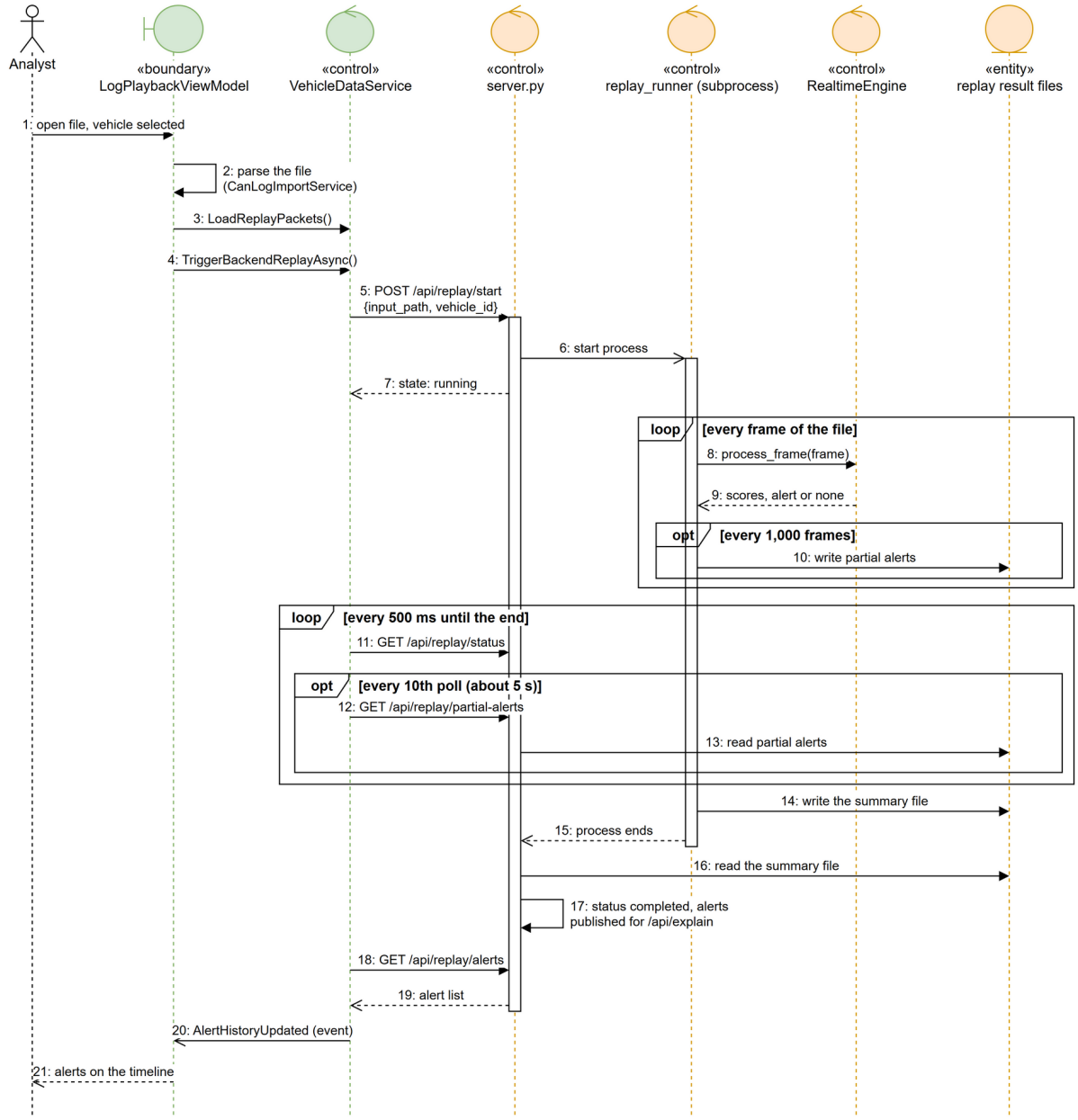


Figure 3.10: Sequence diagram: life cycle of a replay, from the client request to the published alerts.

3.5.2 Explanation of an alert

The explanation engine is a supporting feature, not part of detection. It turns the context of an alert into a text an analyst can read: which signal dominated, the likely cause, and what to check. It is called after an alert is raised and never changes a score, a flag or a severity.

When the analyst selects an alert (Figure 3.11), the backend gathers its context: the measured values and layer scores, the usual behaviour of the identifier, the decoded signals when a DBC file exists, known anomaly patterns, similar past cases and, for Gemini, a short web search. Gemini writes the explanation, or a local Ollama model if Gemini cannot be reached; if neither answers, a rule based template fills the same fields from the measured values (NFR6). The engine records the numbers of a model's answer that do not appear in its context, and every answer carries the name of the engine that wrote it, so that a template is never presented as the work of a language model.

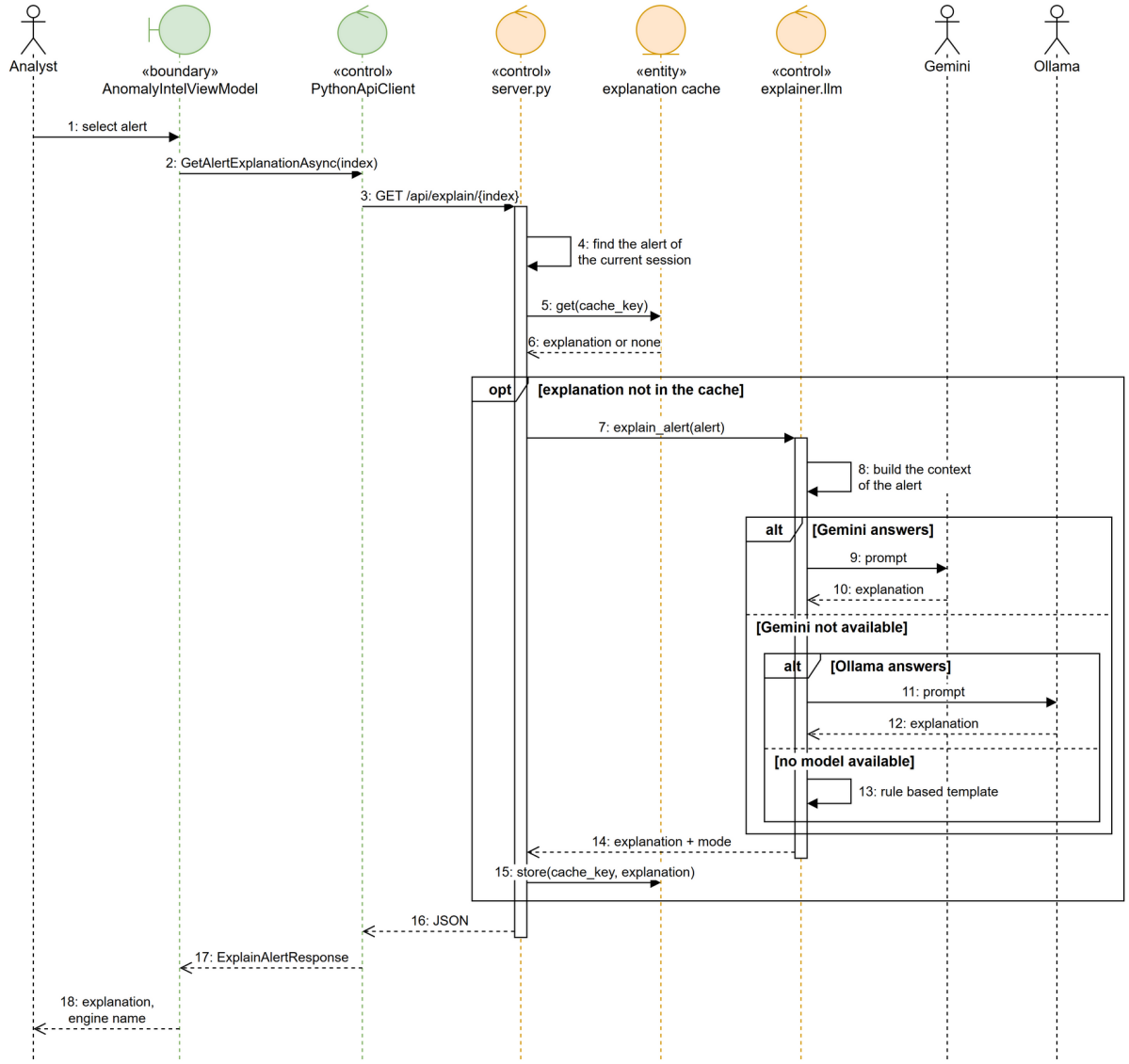


Figure 3.11: Sequence diagram: requesting the explanation of an alert.

3.6 Detection architecture

Frames reach the detection through two paths (Figure 3.12). Replay, simulation and live capture process frames one by one in the runtime engine; the offline analysis receives a whole recording at once. Both use the same features and models, but they turn scores into alerts differently, so the same recording can give different numbers of alerts on each path, and a comparison is only meaningful on one path.

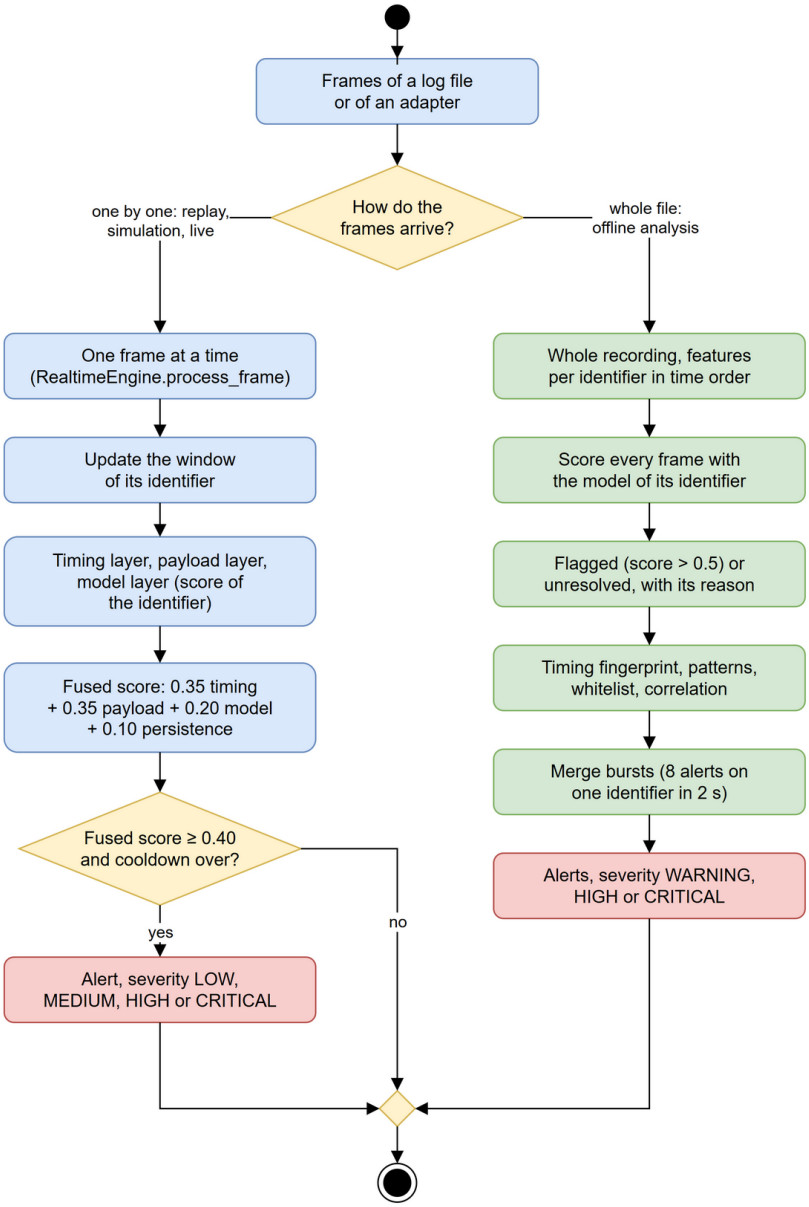


Figure 3.12: Detection pipeline: feature computation per identifier, scoring by the model, additional checks and alerting.

3.6.1 Frame by frame path (live, simulation, replay)

For each frame, the runtime engine computes four layer scores:

- **Timing:** The variation of the time between messages of the identifier and its message frequency, whichever is larger, since a regular flood has almost no variation but a very high rate.
- **Payload:** How abruptly the bytes change from one frame of the identifier to the next.
- **Model:** The normalised score given by the model of the vehicle. A frame that no model can score gets no score here, and its status is reported.
- **Persistence:** How many recent frames had a high combined score, so that a sustained anomaly weighs more than a spike.

These are combined into one fused score with fixed weights, and Table 3.1 gives the alert and severity rules. The weights and thresholds are configuration values, not learned from data.

Table 3.1: Scoring and alerting rules of the two detection paths.

	Frame by frame path (replay, simulation, live)	**Offline analysis**
Score	Fused score, each layer from 0 to 2: 0.35 timing + 0.35 payload + 0.20 model + 0.10 persistence	Normalised score of the model, from 0 to 1
Model call	Every fifth frame of each identifier, the last score reused in between	Every frame
Alert	Fused score ≥ 0.40, at most one alert per identifier and second	Each flagged frame (score > 0.5), annotated by the four checks; more than eight alerts on one identifier within two seconds are merged
Severity	LOW ≥ 0.30, MEDIUM ≥ 0.55, HIGH ≥ 0.80, CRITICAL ≥ 1.00	WARNING below 0.55, HIGH ≥ 0.55, CRITICAL ≥ 0.80; shown in the client as INFO, WARNING and CRITICAL

3.6.2 Offline analysis path

The offline analysis scores every frame with the model of the vehicle. A frame is *flagged* when its score passes the threshold of the model, and a frame that no model can score is counted as *unresolved*, with its reason, instead of normal. Four checks follow: a timing fingerprint of each identifier built from the start of the recording, a frequency analysis that names bursts, frozen payloads and rate spikes, a whitelist of the identifiers seen at the start, and a correlation check between signals linked by a rule, for example a high motor speed with the throttle at zero. Bursts of alerts on one identifier are merged into one summary alert.

3.7 Data and machine learning problem framing

3.7.1 Problem definition

The learning problem is to decide whether a frame looks like the normal traffic of a given vehicle, without labelled attacks for it. A frame is judged within its own context: a battery voltage message is compared with the usual behaviour of that message, not with the traffic as a whole.

3.7.2 Dataset files

All the recordings come from public sources (Figure 3.13 and Table 3.2); none was bought or obtained under a non disclosure agreement. Only Car-Hacking and the separate attack captures of ROAD carry attack labels.



Figure 3.13: Sources of the recordings: CSS Electronics, Oak Ridge National Laboratory (ROAD), Korea University (Car-Hacking) and GitHub (EV-CANlogs).

Table 3.2: Dataset files used in the project.

Dataset	**Vehicles**	**Format**	**Use in the project**
CSS Electronics sample data, including the EV Data Pack [28]	Kia EV6, Nissan Leaf 2019	MF4	Kia EV6: validation of the first pipeline; Nissan Leaf 2019: training and holdout of a final model
ROAD [6]	Real vehicle on a dynamometer	candump log	Ambient recordings only: first corpus, training and holdout of the ROAD ambient model
Car-Hacking [5]	Real vehicle	CSV with labels	Normal rows only, in the first corpus
EV-CANlogs [29]	Jaguar I-PACE, Kia EV6, BMW i3, MG ZS EV, Tesla and others	CSV, candump, Vector	Most of the first corpus; training and holdout of the Jaguar I-PACE and Kia EV6 models
Renault Clio log	Renault Clio	candump log	Only to check that no data leaks between sessions

3.7.3 Supervised or unsupervised learning

A supervised approach would need labelled normal and attack frames for every vehicle. Most of the corpus has no labels, the few labelled attacks come from other vehicles and attack methods, a model trained on known attacks is unlikely to recognise a new one, and recording real attacks on every vehicle was not possible. An unsupervised approach only needs the normal traffic of the vehicle, which the corpus has in plenty.

3.7.4 Candidate algorithms

Table 3.3 compares the algorithms considered for the anomaly score.

Table 3.3: Candidate machine learning algorithms for the anomaly score.

Algorithm	**Advantages**	**Disadvantages**
Isolation Forest [24]	Fast to train, no distance computation, no assumption on the distribution; suited to numeric tabular features.	Harder to read than a rule; needs a decision threshold.
One Class SVM [30]	Well studied, with a clear geometric meaning.	Training cost grows fast with the data; sensitive to its kernel.
Local Outlier Factor (LOF) [31]	Captures local differences of density.	Keeps the training data; its prediction cost grows with it.
DBSCAN [32]	Finds clusters of any shape without fixing their number.	Gives no score for a new point.
Random Forest (supervised)	Very accurate with labelled data.	Needs labelled attacks for each vehicle.
Autoencoder	Can model complex normal patterns.	Needs more data and tuning; harder to explain.
K-Means [25]	Simple and fast.	Not an anomaly detector by itself.

3.7.5 Choice of Isolation Forest

The algorithm had to learn from normal data only, give a result an analyst can understand (“how unusual is this frame”), and score a frame quickly enough for replay and live capture. Isolation Forest meets the first two criteria by construction: it is trained on normal traffic, and an anomalous point is isolated after fewer random splits. DBSCAN was set aside because it gives no score for a new frame, the autoencoder because it is harder to explain and debug, and the supervised Random Forest because no labelled attacks exist for our vehicles. LOF and the One Class SVM remained credible alternatives, so we planned to measure them against Isolation Forest once the final models existed, on the same data and under the same rule.

3.8 Architecture baseline

The elaboration phase ended with an executable architecture, delivered by milestone M1, the live foundation. It connected the two programs end to end before any vehicle model existed: the client started and stopped a live session through new API endpoints, the backend scored a simulated stream frame by frame in the runtime engine, an alert reached the client, and the Dashboard charted the fused score of the last frames. A smoke test checked this chain, from the simulated data to the scoring, the alert and the update of the Dashboard.

This baseline reduced three risks of Table 2.9. Live capture would only differ from replay and simulation by its transport, since all three feed the same runtime engine (R1). Every source would enter the engine as the same frame representation (R7). And the client and the backend were shown to work together through the API, so that the following iterations could add features on both sides without changing how they communicate.

3.9 Conclusion

This chapter presented the technical choices, the architecture of the client and the backend, the domain and data models, the dynamic view of a replay and of an explanation, the two detection paths, the framing of the machine learning problem, and the architecture baseline of the elaboration phase. The next chapter presents the four iterations of the construction phase.

Chapter 4

Construction: Iterations

4.1 Introduction

This chapter presents the construction phase, during which CANvisionNative was built on the architecture baseline of Chapter 3. It first explains how the work of the phase was planned in iterations, then presents the four iterations one after the other: the first detection pipeline and client, its validation and correction on a real recording, the hardware layer and its bench rig, and the rebuild of the vehicle models.

4.2 Iteration planning

The construction phase was divided into four iterations, each one addressing some of the risks of Table 2.9. Each iteration is presented with the same template: its objectives, the use cases it addressed, its design, its implementation, its tests, and an assessment that decided what the next iteration had to do. Table 4.1 gives the period, the goal and the main risks addressed by each iteration.

Table 4.1: Iterations of the construction phase.

Iteration	**Period (2026)**	**Goal**	**Main risks addressed**
1	March to June	First detection pipeline and client (M3 to M8)	R2, R7
2	June to July	Validation and correction on a Kia EV6 recording	R3, R6, R7, R8
3	August to September	Hardware layer and bench rig (M2)	R1
4	September	Model rebuild and evaluation on recordings kept out of training	R3, R4, R5, R6

4.3 Iteration 1: first detection pipeline and client

4.3.1 Objectives

The first iteration delivered milestones M3 to M8 on the architecture baseline: models trained for several vehicles and driving states (M3), the explanation engine (M4), the real time charts (M5), session recording (M6), the integration of the client and the backend (M7), and the decoding pipeline for offline and live traffic (M8). At its end, an analyst could import a log, analyse and replay it, review and explain its alerts, and export them.

4.3.2 Use cases addressed

This iteration addressed the use cases *Import log*, *Select vehicle*, *Replay log*, *Start live simulation*, *Detect anomaly*, *Record session*, *Review alerts*, *Request explanation*, *Export report* and *Configure settings*, which together cover the work of the analyst on a recorded log. The use cases *Connect hardware*, *Check model status* and *Describe capture* were addressed in later iterations.

4.3.3 Design: a first model based on driving states

The first design routed each frame through a driving state. Fourteen rolling features were computed for every identifier, a K-Means model shared by all vehicles classified the state as parking, idle or driving (Figure 4.1), and the frame was scored by the Isolation Forest of that state and vehicle. When no forest matched, the engine fell back to a generic model, then to a heuristic. These models were trained on more than seven million frames from sixty six sources of the corpus of Table 3.2.

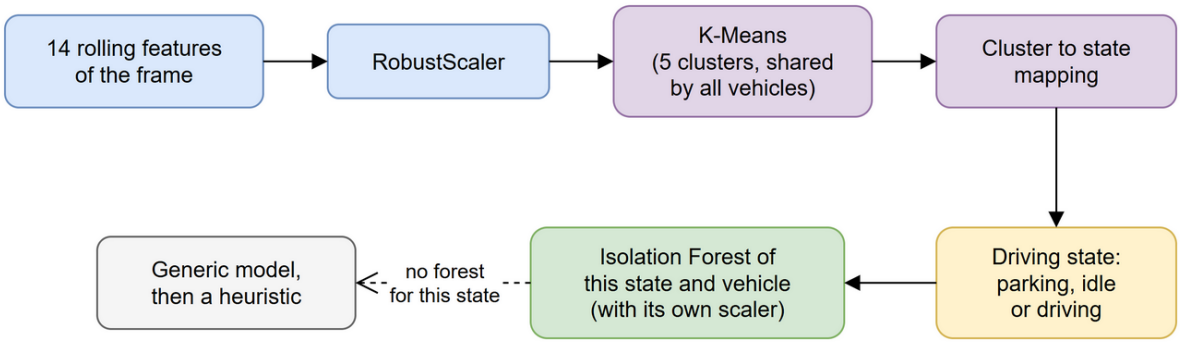


Figure 4.1: Scoring path of the first design: K-Means on fourteen rolling features chose the driving state, which selected the Isolation Forest.

4.3.4 Implementation

The iteration implemented the backend services below on the architecture baseline, then the screens and exports of the client.

4.3.4.1 Parser and common frame representation

One dispatch function turns every supported format into the same stream of (timestamp, identifier, payload) frames:

- **.csv:** Generic columns, or the SavvyCAN columns with timestamps in microseconds.
- **.mf4:** ASAM MF4 [26] binary logs, read with asammdf.
- **.asc:** The Vector text format.
- **.log:** The candump and SocketCAN format, including its timestamped variant.
- **.txt:** The PCAN text format, with candump parsing as a fallback.

The offline analysis called this function from the start. The replay engine, however, still had its own loader, which only understood CSV; this gap was found and closed in the second iteration (Fix 4).

4.3.4.2 Replay engine

The replay process writes partial results during the run, so that the client can show progress, and a summary with the full alert list at the end. In the client, the Log Playback screen plays the frames at the pace of their own timestamps, multiplied by the chosen speed; a first version played one frame per screen refresh, which ran a bus of 3,000 frames per second about fifty times slower than real time.

4.3.4.3 Vehicle profiles

Two registries describe the vehicles. The vehicle profiles (Table 4.2) give a bitrate and the DBC files that decode the frames; the last one was written for the bench rig in the third iteration. The second registry lists the trained models of each vehicle. The two lists do not match one to one: the Jaguar I-PACE has a model but no DBC file, while the BMW i3 has a DBC file but only a model of the first design.

Table 4.2: Vehicle profiles with DBC decoding.

Identifier	Display name	Bitrate
nissan_leaf	Nissan Leaf (2019, 40/62 kWh)	500 kbps
hyundai_ioniq5	Hyundai Ioniq 5	500 kbps
hyundai_kia	Hyundai / Kia (generic)	500 kbps
skoda_enyaq	Škoda Enyaq iV (VW MEB)	500 kbps
vw_skoda_audi	VW / Škoda / Audi (generic)	500 kbps
renault_zoe	Renault Zoe	500 kbps
tesla_model3	Tesla Model 3	500 kbps
kia_ev6	Kia EV6	500 kbps
bmw_i3	BMW i3 (2017, 60 Ah REX)	500 kbps
bench_rig	Arduino bench rig	500 kbps

4.3.4.4 Explanation module

The explanation module implements the design of Section 3.5.2. The context of an alert is built in the backend, and the providers are called in turn through their HTTP interfaces. Each explanation is added to a history store, so that later similar anomalies can refer to it, and cached during a run, so that the same alert is not explained twice.

4.3.4.5 Client and exports

The client implements the seven screens of Section 3.3.3, presented in Chapter 5. Anomaly Intel exports the alert list as a CSV file or as a PDF report built with PDFsharp, both read from the list on screen.

4.3.5 Tests

Each milestone was checked by running it on recorded logs and by a demonstration in the client before the next one started. The systematic validation of the whole pipeline on a real recording was left to the second iteration.

4.3.6 Assessment

All the milestones M3 to M8 were delivered, and the first pipeline worked from the import of a log to the export of its alerts. It had not yet been checked against facts known independently, which was the goal of the second iteration.

4.4 Iteration 2: validation and correction on the Kia EV6

4.4.1 Objectives

The second iteration validated the first pipeline on a real recording, found its defects and corrected them. One rule guided it (NFR2): a claim had to be backed by reproducible evidence before any code was changed. Tuning a threshold or retraining blindly were not accepted as fixes. A defect counted as found only once it could be triggered on demand and explained in the code, and as fixed only once the same test, run again, gave the expected result.

4.4.2 Use cases addressed

The validation followed the path of an analyst on a recording, through the use cases *Import log*, *Select vehicle*, *Replay log* and *Detect anomaly*, then *Request explanation* and *Export report* on the alerts that the replay produced.

4.4.3 Design of the validation

The validation took three passes (Figure 4.2): an audit of the interface, an analysis of the root causes of the alerts, and the correction of the defects, each verified again, followed by a full run of the corrected pipeline.

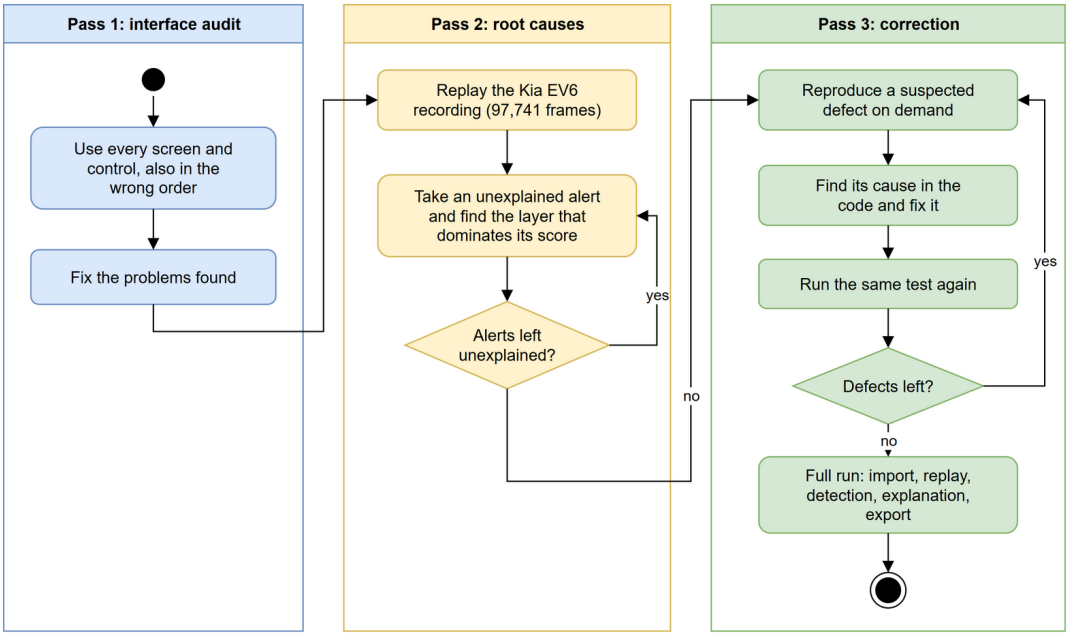


Figure 4.2: Passes of the validation of the first pipeline.

The iteration used a one hour recording of a Kia EV6 (`00000158-64CBC290.MF4`) from the CSS Electronics EV Data Pack (Table 3.2), captured by a CANedge logger that actively polled the ECUs with UDS requests. After conversion, it contains **97,741 frames**. It was the only available recording that came from an unmodified production vehicle, was long enough for measurements before and after a fix, had a frame count that could be checked against the metadata of the file, and belonged to a vehicle with its own profile and model. A candump log of a Renault Clio, from another vehicle and logger, served to check that no data leaks between sessions.

4.4.4 Implementation: audit and corrections

The iteration started with an audit of the interface, fixed the conversion of the recording, then analysed the alerts and corrected the defects they revealed.

4.4.4.1 Interactive interface audit

Before looking at the data, we drove the running client through interface automation: every screen, the import of valid, empty, corrupted and very large files, the full life cycle of a replay, and actions done in the wrong order or several times. This revealed an implausible latency shown on three screens, a rendering problem caused by a font substitution, and destructive actions that ran without confirmation, all fixed before the data driven validation.

4.4.4.2 Conversion of the MF4 file

Loggers of the CANedge type spread the traffic of an MF4 file over several channel groups, most of them empty templates. Our first conversion used the default view of the library, which shows only the first channel group and silently drops the others, so it returned a small part of the traffic. The conversion now goes through every group and merges those that hold data; checking the number of frames against the metadata of the file confirmed the fix and gave the 97,741 frames used in this iteration.

4.4.4.3 Root cause analysis of the alerts

Before any correction, the replay produced **15,945 alerts**, an alert rate of 16.3% that made the system hard to use. Instead of raising the threshold, we attributed every alert to the layer that dominated its score (Table 4.3).

Table 4.3: Root causes of the 15,945 alerts of the first pipeline.

Dominant cause	**Share**	**Explanation**
Machine learning layer	61.42%	Wrong model selected, and wrong driving state (Fixes 1 and 2)
Timing layer	31.51%	Irregular intervals of the UDS polling, which gives a very high alert rate on the polled identifiers by construction
Payload layer	7.07%	Real variation of the sensor values in a battery telemetry message
Total explained	**100%**	

The machine learning share pointed at the defects in model selection and state classification below; the timing and payload shares were explained by the UDS polling and by real sensor variation, so they were not treated as defects. Four defects came from this analysis, and two more appeared when the corrections were checked through the application rather than through the backend API alone. Each is presented as problem, investigation, root cause, fix, evidence and result.

4.4.4.4 Fix 1: vehicle model selection

- **Problem:** Choosing a vehicle in the client had no visible effect: every replay seemed to score the frames the same way.
- **Investigation:** We followed the vehicle identifier from the selector of the client, through the REST call and the replay process, down to the scoring call.
- **Root cause:** The scoring step never received the vehicle chosen by the analyst. The replay engine created its scoring component without telling it which vehicle was active, so every replay was scored with the generic model.

- **Fix:** We passed the vehicle identifier through every stage that sets up scoring: the replay engine, the offline analysis, the live hardware interface, the backend endpoint and the whole call chain of the client down to the selector.
- **Evidence:** We read the command line of the replay process in the process list of the operating system and confirmed that it started with the chosen vehicle. We also confirmed that the vehicle list of the backend returned the same identifier as the folder of the trained model on disk.
- **Result:** Every replay now scores the frames with the model of the chosen vehicle, as required by FR2.

4.4.4.5 Fix 2: driving state classification

- **Problem:** The state classifier reported **99.93%** of the recording (97,677 of 97,741 frames) as “parking”, although the recording is an hour of active driving.
- **Investigation:** We compared, field by field, the feature vector and the scaling given to the classifier with the feature list and the scaler used to train the K-Means model.
- **Root cause:** Two defects added up. The vector given to the classifier was built from a list of 11 features meant for the scoring model, while the K-Means model had been trained on 14 features, so every call raised the error “expected 14 features, got 11”. A very broad error handler caught this error and always returned “parking”, which made the failure constant and therefore invisible unless someone reproduced the exact error on purpose.
- **Fix:** We corrected the feature list to the 14 features in the order used at training, and added the scaling step that was missing before the classification.
- **Evidence:** After the fix, the same recording was classified as **99.92% “driving”** (97,667 of 97,741 frames), with 64 frames “unknown” and 10 “parking”, a credible result for an hour of driving.
- **Result:** The Isolation Forest of the right state is now used for almost every frame. This fix explains most of the improvements reported in Section 4.4.5.

4.4.4.6 Fix 3: attribution of the reason

- **Problem:** The layer shown to the analyst as the “dominant” cause of an alert did not match what the fused score had actually used.
- **Investigation:** We traced the displayed label back from the API response to the internal reason string, and compared it with the real inputs of the fusion.



- **Root cause:** The label took the largest of all the values of the reason string, including two fields kept only for diagnosis: a raw frequency, sometimes in the thousands of hertz, and a raw entropy. These two values are much larger than the four real inputs of the fusion, so they almost always won and the displayed cause was wrong. The same mistake had been written independently six times in four files.
- **Fix:** We restricted the comparison, in all six places, to the four fields that really feed the fused score: timing, payload, machine learning and persistence.
- **Evidence:** By inspecting the inputs of the fusion component, we confirmed that it only uses the normalised layer scores, persistence being computed inside it. The wrong label had therefore never changed the detection itself, only its explanation. After the fix, the alert feed named the right layer, for example the payload layer when it really dominated.
- **Result:** The cause shown for an alert now matches the real inputs of its score everywhere, which makes both the alert list and the explanations built from it more trustworthy.

4.4.4.7 Fix 4: one parser for replay

- **Problem:** Some formats that the file dialog accepted gave empty or meaningless replays.
- **Investigation:** We compared the loading code of the replay engine with the shared loader of Section 4.3.4.1, which the offline analysis already used without problems.
- **Root cause:** The replay engine had its own loading code, which called a generic CSV reader directly. Any .log, .asc or .txt file chosen in the dialog was read as a broken CSV file.
- **Fix:** The replay engine now calls the shared loader, and its own CSV only code was removed.
- **Evidence:** The corrected CSV path gave exactly the same output as before on all 97,741 frames, and small candump and Vector test files, which had produced timestamps and identifiers equal to zero, were parsed correctly.
- **Result:** Replay and offline analysis share one parsing path for every format, as required by NFR4.

4.4.4.8 Fix 5: polling timer left stopped

- **Problem:** After a log had been imported from the Home screen, no later replay could deliver its alerts to the Anomaly Intel screen until the application was restarted.
- **Investigation:** The defect only appeared through the client, never through direct calls to the backend, which is why the backend tests behind Fixes 1 to 4 missed it and it only showed up during the end to end run.

- **Root cause:** Importing a log from the Home screen called a method that stopped the background polling timer of the client before its own analysis, and nothing started it again.
- **Fix:** The timer is now started again every time a replay is triggered, whatever the screen that triggered it.
- **Evidence:** A replay started after an import from the Home screen delivered its alerts to the Anomaly Intel screen again, like a replay started without a previous import.
- **Result:** Alerts now reach the analyst whatever the order of the screens used, a defect that a test of the backend alone could not have found.

4.4.4.9 Fix 6: stale explanation cache

- **Problem:** A request for the explanation of the alert displayed in the client could return the text of another, unrelated alert.
- **Investigation:** We replayed two datasets from different vehicles, the Kia EV6 recording and the Renault Clio log, one after the other, and compared the explanations returned for the same alert index in both runs.
- **Root cause:** The explanation endpoint looks up an alert index in a shared list that lives as long as the process. Before the fix, only the offline analysis and the injection of test anomalies filled this list, never a finished replay. An explanation could therefore be looked up in data left by an earlier analysis, or, before a replay finished, in data left by an even older session.
- **Fix:** Two changes closed the problem. When a replay finishes, its alerts are now published into the same shared list as the other two paths. When a replay starts, this list is cleared at once, so that nothing from a previous session can be found, even before the new replay ends.
- **Evidence:** For every alert index we checked, the explanation matched the identifier and the score of that alert in both datasets, with no trace of the other dataset. A second test filled the shared list with the Renault Clio data, started the Kia EV6 replay and asked for an explanation before the replay finished: the endpoint correctly returned no explanation instead of the old data.
- **Result:** An explanation now always belongs to the alert shown on screen, during and after a replay, which supports the trust required by NFR3.

4.4.5 Tests: measurements before and after the corrections

Table 4.4 and Figure 4.3 compare the measurements on the same recording before any correction and after the four data driven fixes.

Table 4.4: Before and after the corrections, on the Kia EV6 recording (97,741 frames).

Metric	**Before**	**After**
Total alerts	15,945 (16.3%)	11,971 (12.25%)
Replay duration	685 s	502.2 s
Throughput	about 143 frames/s	about 194.6 frames/s
Driving state	99.93% parking	99.92% driving
Saturated machine learning score (≥ 0.999)	77.6% to 78.6%	0.0%

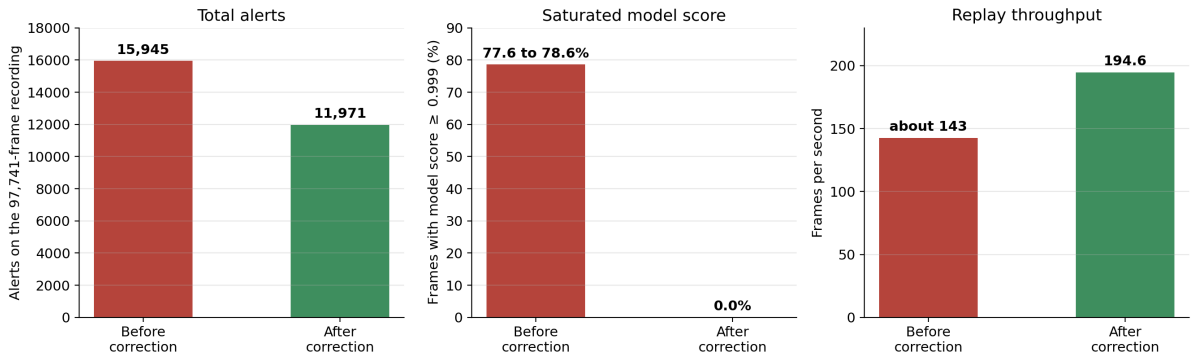


Figure 4.3: Alerts, saturated model score and replay throughput on the Kia EV6 recording, before and after the corrections.

Fewer alerts and a higher throughput are side effects of fixing real defects, not of a threshold change: the right model of the vehicle replaced a saturated generic path. Figure 4.4 shows the evidence of Fix 2.

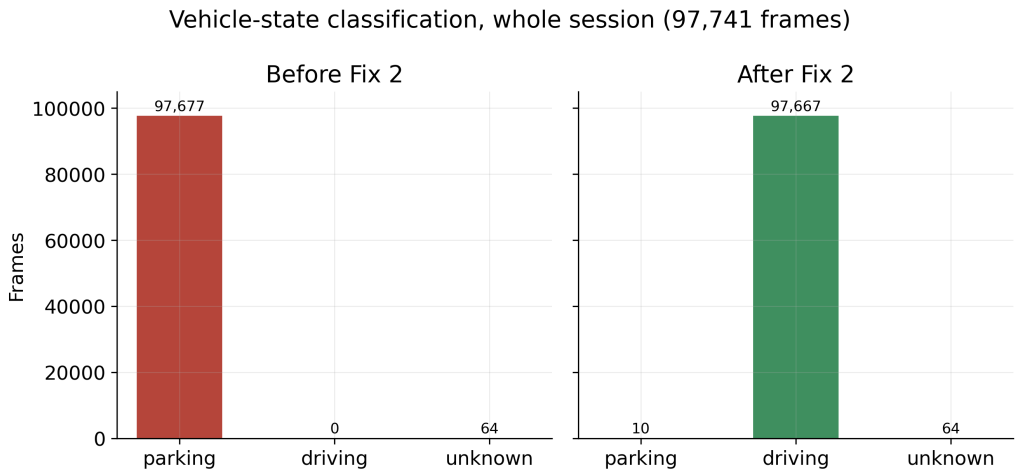


Figure 4.4: Driving state of the whole recording, before and after Fix 2.

Figure 4.5 breaks down the 11,971 remaining alerts. The right chart also supports Fix 3: the raw frequency and entropy no longer appear among the dominant layers.

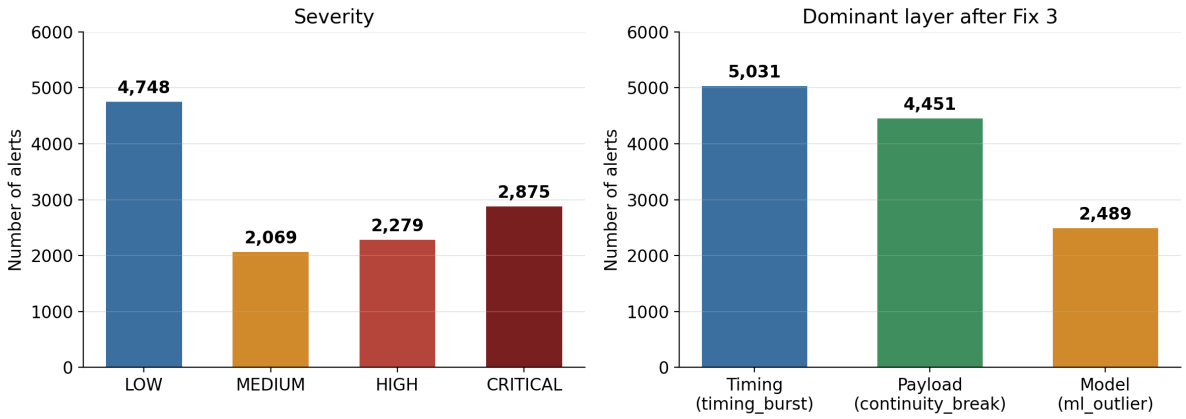


Figure 4.5: Severity (left) and dominant layer after Fix 3 (right) of the 11,971 alerts produced by the corrected first pipeline.

4.4.6 Assessment

Table 4.5 summarises the six defects.

Table 4.5: Defects corrected during the second iteration.

#	Defect	Status
1	Vehicle model never used for scoring	Fixed, verified
2	Driving state always classified as “parking”	Fixed, verified
3	Diagnostic fields dominating the displayed cause (6 places)	Fixed, verified
4	Replay formats other than CSV bypassing the common parser	Fixed, verified
5	Polling timer left stopped after an import	Fixed, verified
6	Explanation cache returning data from another session	Fixed, verified (two changes)

All six defects were silent: none crashed or raised an error, and each appeared only when what the system reported was compared with a fact known independently, such as a vehicle being driven or a file being valid. The corrections reduced the alerts by a quarter without touching the threshold. The corrected pipeline worked, but the iteration exposed three weaknesses of its design. Two models in a chain meant that a wrong state selected the wrong forest, the source of two of the six defects. One forest per state mixed identifiers sent every 10 milliseconds with identifiers sent every second. And the fallbacks hid missing models. It was also not always possible to prove which data and settings had produced a model file. These weaknesses led to the rebuild of the models, planned as the fourth iteration, after the hardware milestone.

4.5 Iteration 3: hardware layer and bench rig

4.5.1 Objectives

The third iteration delivered milestone M2: live capture from a physical CAN adapter (FR9), and a way to test it without a vehicle, since none was available.

4.5.2 Use cases addressed

This iteration addressed the use case *Connect hardware*, together with *Detect anomaly*, applied to the frames of the live bus, and *Record session*, which saves a live session to a log file.

4.5.3 Design

The hardware layer wraps python-can, so it accepts the adapters python-can drives, such as slcan serial adapters, PEAK PCAN or SocketCAN, plus a virtual bus for tests. Every received frame goes to the same runtime engine as replay, built for the model chosen by the analyst, and is decoded with the DBC file of the vehicle when it has a profile. A profile can declare its DBC file complete, meaning that it defines every identifier of the bus, and live capture then flags any other identifier; since the DBC files of real vehicles are partial, only a profile written for a known test bus can do so. Because the engine is shared, a bench rig test without a vehicle is meaningful for the part specific to live capture, its transport, while the engine behind it is the one that scores replays.

4.5.4 Implementation

Connecting an adapter builds a live engine for the model chosen by the analyst, and disconnecting stops the scoring. When no frame arrives for a few seconds, the link is reported as lost. Frames are decoded with the DBC file of the selected profile and, with a complete profile, a frame on an identifier outside the file raises an alert on the whitelist layer.

Our bench rig replaces the vehicle. Two Arduino Uno boards, each with an MCP2515 CAN module, are joined by a two wire bus terminated with 120 Ω resistors, at 500 kbit/s. The bridge board is connected by USB and runs a small firmware that speaks the slcan protocol expected by python-can. The simulator board sends a speed frame every 20 milliseconds and a state of charge frame every 200 milliseconds, about 55 frames per second, and, from the 35th second after power on, a frame every twenty seconds on an identifier that should not exist (0x666), which the profile of the rig leaves out on purpose.

4.5.5 Tests

Live capture was tested on the bench rig in two steps. First, a short script read the bus through python-can: the three identifiers arrived with the values of the firmware, a speed of 60 km/h and a state of charge of 78%, which showed that both boards, the wiring and the serial link worked. Second, the rig was connected from the application (slcan, 500 kbit/s), so that the frames went through the hardware layer into the runtime engine.

This second step revealed two defects of the hardware layer, which we corrected:

- **One bad line closed the session.** After about eight minutes and 16,304 frames received, a single malformed line on the serial link raised an error that ended the connection. One bad frame does not mean that the link is dead, so the hardware layer now skips such a line, logs it and keeps reading. After this change, the connection stayed up and received frames at about 55 per second, the rate of the firmware.

- **Old data stayed on screen after the rig was unplugged.** When the boards were disconnected, the link was still reported as up, because its status only changed on a hard error, and the live screens kept showing the last frames received, hours later. The link is now reported as lost when no frame has arrived for five seconds, disconnecting clears the buffers of live data, and each live update says whether it comes from the hardware or from the simulator, so that the client can show which one the analyst is looking at.

Two more gaps concerned the interpretation of live frames. First, the frames were not decoded correctly. The rig had been connected under the profile of a real vehicle, whose DBC file does not define 0x316 or 0x3F9, and the live signal panel read the frames with a signal map shared by all vehicles, which gave them the meaning of unrelated messages from another manufacturer. Second, the frame on the unknown identifier was never raised as an alert, and it could not have been: the whitelist of Section 3.6 belonged to the offline analysis, and live capture had no check of the identifiers at all. We corrected both. The rig now has its own vehicle profile and DBC file, and the live screens use the decoding of the selected profile first, so the live values sent to the screens become a speed of 60 km/h and a state of charge of 78%. The profile of the rig declares its DBC file complete, so live capture flags 0x666 on its first frame, without any learning window. Automated tests send the three frames of the rig through the virtual bus of python-can and check the decoded values, the alert on 0x666, and the absence of such alerts for a profile whose DBC file is partial.

4.5.6 Assessment

The transport of live capture, from the bus to the runtime engine, works on the bench rig. This test does not measure detection: the traffic of the rig is synthetic and much simpler than the traffic of a vehicle. We have not yet repeated the run on the physical rig after the last corrections, and the test was made before the final models of the fourth iteration were promoted. Risk R1 remains open: live capture has not been tested on a vehicle.



4.6 Iteration 4: model rebuild

4.6.1 Objectives

The fourth iteration rebuilt every vehicle model to remove the weaknesses found in the second iteration: one model per CAN identifier instead of a chain of models, a record of how each model was built, and an evaluation on a complete recording kept out of training before any model enters production. It also measured the speed of the final models and compared Isolation Forest with the other candidate algorithms.

4.6.2 Use cases addressed

This iteration addressed the use cases *Select vehicle* and *Check model status*, since every model now has a status that decides what it may be used for, *Detect anomaly* with the new models, and *Describe capture*, which tells the analyst whether the selected model was trained or tested on the loaded file.

4.6.3 Design

4.6.3.1 Final design: one model per CAN identifier

In the final design, each vehicle has one Isolation Forest per CAN identifier, with no driving state and no fallback: a frame is scored by the model of its identifier, or not at all.

- **Features:** Nine features are computed over a short window of recent frames of the identifier: the time since the previous frame with its rolling mean and standard deviation, the byte difference with the previous frame with its rolling mean, standard deviation and maximum, the number of changed bytes, and the payload entropy. A feature that never changes for an identifier is dropped for it. The same code computes the features for training, offline analysis and live scoring.
- **Unscored frames:** An identifier needs 1,000 training rows to get a model; below that it is *sparse*, an identifier whose features never vary is *unmodelled*, and one absent from training is *unseen*. Their frames, and the first frames of any identifier (*warmup*), are reported as unresolved with their reason, so the analyst sees how much of a recording the models covered.
- **Threshold:** Each model has its own threshold, set so that 2% of its training rows are flagged. The raw score is mapped to a normalised score: 0 at the median of the training rows, 0.5 at the threshold, closer to 1 as the frame becomes more anomalous. The 2% value was fixed before any test recording was scored and never changed afterwards.

Table 4.6 lists the parameters, also recorded in the manifest of every model.

Table 4.6: Parameters of the final models.

Forest	scikit-learn Isolation Forest, 200 trees, random seed 42, no feature scaling (a tree splits on one feature at a time)
Window and warmup	The last 10 frames of the identifier; its first 10 frames are not scored
Minimum data	1,000 training rows per identifier
Threshold	With n training rows and $k = \lfloor 0.02\,n \rfloor$, the $(k+1)$th smallest raw score; a frame is flagged below it

4.6.3.2 Training and promotion pipeline

Figure 4.6 shows how the models are produced. The corpus of each vehicle is split by recording, not by frame: at least one complete recording is kept aside as a *holdout*, never used to fit the model or set its threshold, because two frames of the same recording are too similar for a frame level split to say anything about a new recording. The SHA-256 fingerprint of each file guarantees that no recording is used on both sides.

A training run writes the models into a staging folder with a *manifest* that records the code versions, the Git commit, the parameters, the threshold, median and row count of each identifier, and the SHA-256 fingerprints of the model file and of every recording. When the backend loads a vehicle, it checks that the model file and the manifest agree and that no recording is on both sides, and refuses the model otherwise. After the holdout evaluation, the promotion step copies accepted models to production and archives the previous files.

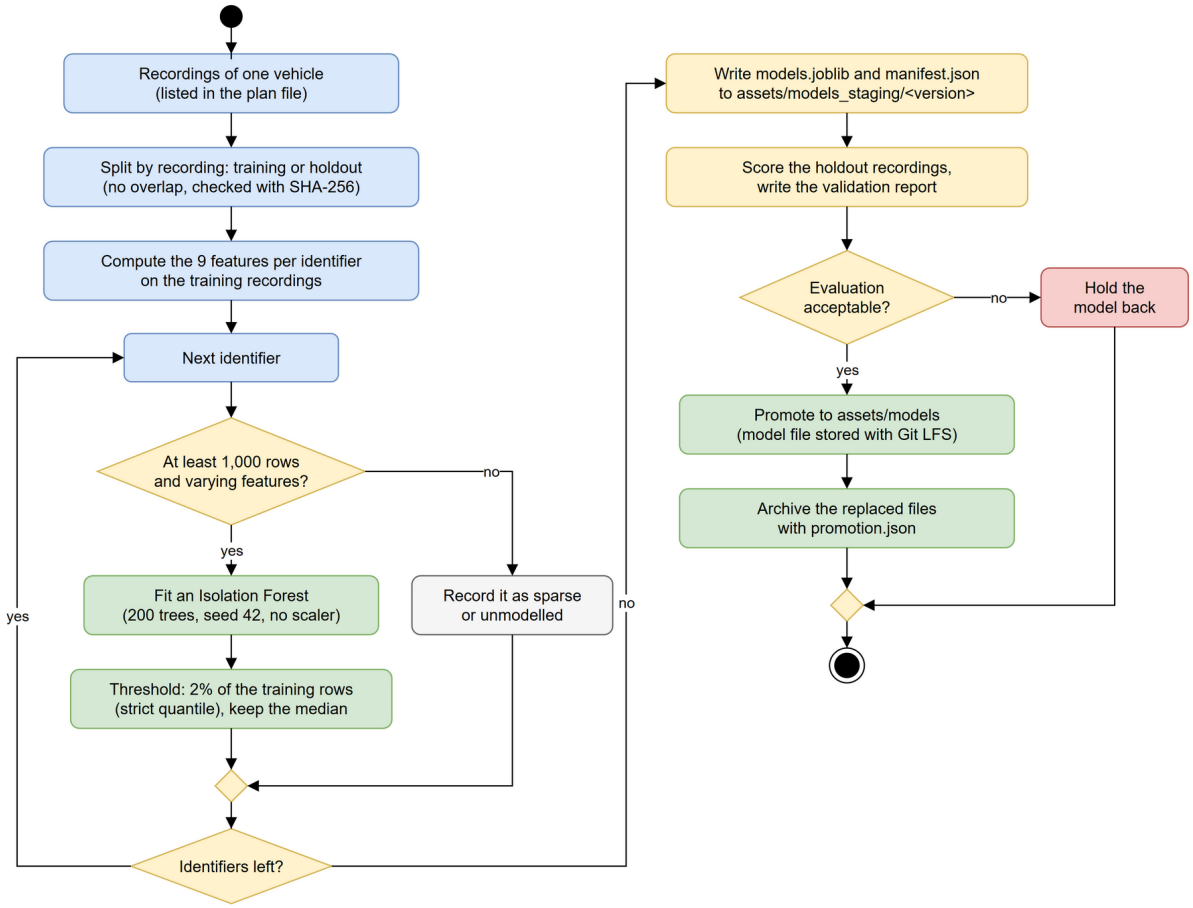


Figure 4.6: Training pipeline, from the recordings of a vehicle to a model in production.

4.6.3.3 Model status

Some vehicles have a single recording, so no holdout exists, and some models fail their evaluation. Each model therefore has a status that decides what it may be used for (Table 4.7), and the client shows it next to each model. Figure 4.7 shows the life cycle of a model under the new pipeline as a state diagram.

Table 4.7: Status of a model and what it allows.

Status	**Meaning and use**
Evaluated on a holdout	Final model in production. Used offline and live.
Not validated, no holdout	A final model exists, but the vehicle has only one recording. Refused offline; live capture uses the model of the first design, with a visible label.
Held back	Failed its evaluation. Refused offline and live.
No final model	No model could be built under the new rules. Refused.
Legacy	Only a model of the first design exists. Refused offline; accepted live with a visible label.
General	The general model of the first design, for a vehicle without its own model.

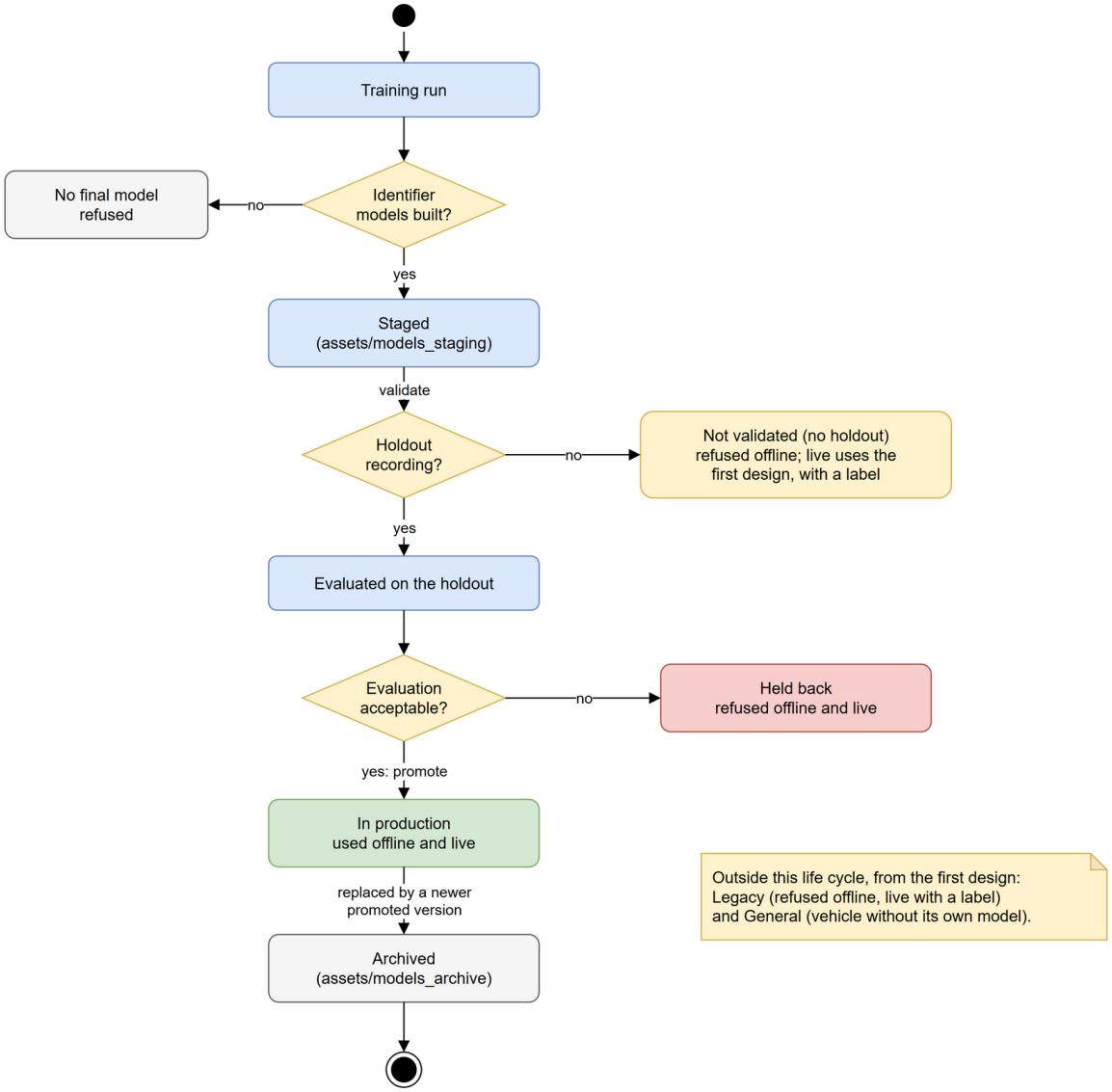


Figure 4.7: State diagram: life cycle of a vehicle model.

4.6.4 Implementation

The training pipeline is a set of Python scripts driven by a plan file that lists the training and holdout recordings of each vehicle. Its commands check the corpus and print the split without fitting anything, train a version into the staging folder, validate a staged version, and promote it. A run produces a validation report, read before any promotion; the promotion script refuses a held back vehicle and archives the files it replaces. The model files are stored with Git LFS.

At run time, one resolver gives the status of every model to the offline analysis, the live capture and the model list of the client, so they never disagree, and a model that may not be used is refused with an explicit reason instead of being replaced. The live engine of the hardware layer goes through the same resolver, so a held back or unknown model is refused for live capture too.

The manifests also describe the recordings. The dataset card of the client sends the fingerprint of the loaded file to the backend, which looks it up in the manifest of the selected model and

answers whether the model was trained on this file, tested on it, or never saw it. For the signals of a capture (FR12), the backend keeps the decoded signals that really change when a DBC file exists, and ranks speed, state of charge, voltage, current and temperatures first; without one, it shows the raw payload bytes that change smoothly, as a physical value does, leaves out constants and rolling counters, and labels every raw channel as undecoded.

4.6.5 Tests

4.6.5.1 Evaluation protocol

Each final model was scored on every frame of its holdout recording, counting the frames scored, the frames flagged, and the frames that could not be scored, with their reason.

The holdout recordings carry no attack labels; we assume that they contain normal traffic but cannot prove it. We therefore report a *flagged rate*, the share of scored frames that the model flagged, and not a false positive rate, a precision or a recall. If a holdout recording is entirely normal, every flagged frame in it is a false alarm and the flagged rate is the false alarm rate; if it is not, the flagged rate also contains real anomalies. Since each threshold flags 2% of the training frames by construction, a flagged rate close to 2% on a new recording means that the model sees this recording roughly as it saw its training data.

4.6.5.2 Holdout results

Table 4.8 gives the results of the four models put into production. Together, they were trained on 23 recordings (3,436,061 frames) and evaluated on 1,130,500 frames of other recordings.

Table 4.8: Final models scored on their holdout recordings.

Vehicle	ID models	Training frames	Holdout frames	Scored	Flagged / scored	Rate
Jaguar I-PACE	46	233,598	50,361	82.6%	1,249 / 40,477	3.09%
Kia EV6	161	448,198	438,170	98.4%	11,109 / 429,524	2.59%
Nissan Leaf 2019	12	1,317,070	491,969	100.0%	2,951 / 491,797	0.60%
ROAD ambient	77	1,437,195	150,000	99.0%	8,982 / 147,485	6.09%

The holdout recordings are `canbus_turningon_quickly.csv` for the Jaguar I-PACE, `Vehicle everthing on.asc` for the Kia EV6 (the spelling is the one of the original file), `00000001-61D9D9BA.MF4` for the Nissan Leaf 2019, and the first 150,000 frames of `ambient_highway_street_driving_long.log` for the ROAD vehicle. The *Scored* column is the share of frames that belong to an identifier with a model, after warmup. The Jaguar I-PACE has the lowest coverage: on its holdout file, 8,451 frames belong to sparse identifiers, 1,368 are warmup frames and 65 belong to unseen identifiers, all reported as unresolved rather than normal.

We read these results as follows.

- **Nissan Leaf 2019 (0.60%).** The model flags fewer frames on its holdout than on its training data. The holdout comes from the same logger as the training recordings, and the model sees it as ordinary traffic.

- **Jaguar I-PACE (3.09%) and Kia EV6 (2.59%).** Both rates are close to the 2% of the calibration, slightly above it, which is what we expected for a new session of the same vehicle.
- **ROAD ambient (6.09%).** This rate is three times the calibration level. Most of the difference comes from one identifier, 0x193: its payload never changes in any training recording, but it does change on the highway recording used as holdout, and 58% of its holdout frames are flagged. This is a difference of coverage between training and holdout, not a problem of threshold. We did not change the threshold after seeing the holdout, since this would have turned the holdout into training data.

4.6.5.3 Models we did not put into production

The same protocol showed which models not to trust. The MG ZS EV model flagged 43.60% of the scored frames of its holdout, because about 78% of its holdout frames belong to identifiers absent from its only training recording; it was held back. Three other model families (the Hyundai Ioniq 5, a Tesla and a Kaggle dataset of normal traffic) have a single recording, so their models are marked as not validated and refused by the offline analysis, and for two vehicles no final model could be built (Table 4.7).

4.6.5.4 Consistency checks

For every identifier model, the threshold recomputed from the training rows matched the manifest (rows, k, threshold, flagged rows and median), and the fingerprints confirmed that no recording is both in the training set and in the holdout of a vehicle.

4.6.5.5 Speed of the final models

A benchmark replayed the first 10,000 frames of each holdout through the runtime engine, as live capture would (Table 4.9).

Table 4.9: Speed of the live path with the final models, on the first 10,000 frames of each holdout. The model call is given as mean and 99th percentile.

Vehicle	Bus (frames/s)	Processed (frames/s)	Real time factor	Model call (ms)
Jaguar I-PACE	2,990	262	0.09	23.6 / 32.7
Kia EV6	448	224	0.50	23.5 / 33.8
Nissan Leaf 2019	98	205	2.10	23.8 / 34.6
ROAD ambient	2,391	219	0.09	23.7 / 34.7

One model call costs about 24 milliseconds, so even with the model called on every fifth frame, the live path processes about 200 to 260 frames per second. It keeps up with the Nissan Leaf (about 100 frames per second) but not with the Jaguar I-PACE or the ROAD vehicle (thousands

per second), and reaches half of the Kia EV6 rate. The offline analysis scores whole batches: the 491,969 frames of the Leaf holdout took 187 seconds, and the Jaguar file 16 seconds in the backend and about one minute in the application. NFR1 is therefore met offline, and live only for slow buses.

4.6.5.6 Comparison with the other candidate algorithms

As planned in Section 3.7.4, we measured LOF and the One Class SVM against Isolation Forest under the same conditions (Figure 4.8). The four vehicles with a final model keep their production split, and every identifier with a final model is fitted three times, on the same features and the same training rows: at most 5,000 evenly spaced rows per identifier, because the One Class SVM does not scale to the largest ones. The three models get the threshold rule of the final models and score the holdout recording, which none of them saw.

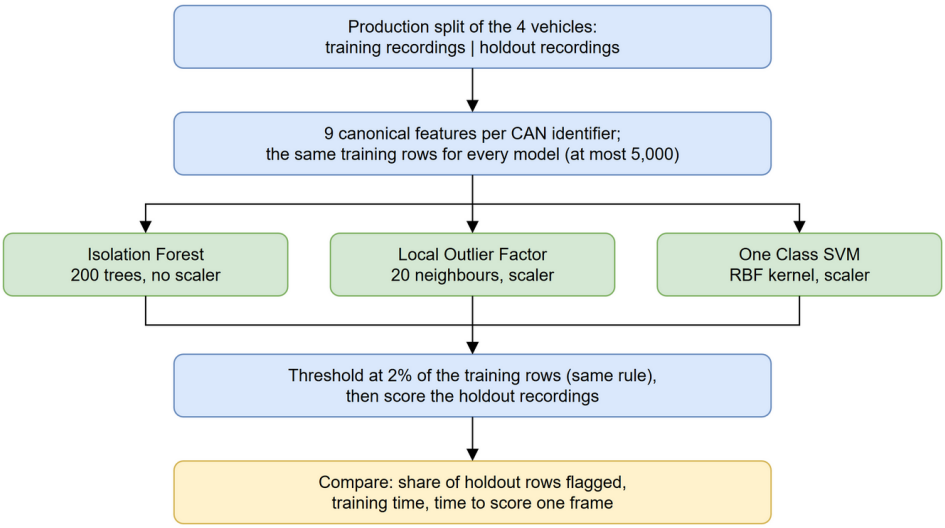


Figure 4.8: Pipeline of the comparison: the same split, features, training rows and threshold rule for the three algorithms.

There are no attack labels, so the measure comes from the holdout recordings, which we assume to be normal traffic (Section 4.6.5.1): every flagged row is then a false alarm, and a model whose boundary generalises flags about the 2% of its calibration. Isolation Forest flags the fewest normal rows on average, 3.2% against 5.1% for LOF and 6.5% for the One Class SVM, and has the lowest worst case, 6.4% on ROAD against 13.1% and 14.0% on the Jaguar I-PACE (Figure 4.9). It is not the best everywhere: LOF does as well on the Kia EV6 and better on ROAD. Trained on these capped rows, Isolation Forest stays close to the final models trained on all rows (Table 4.8).

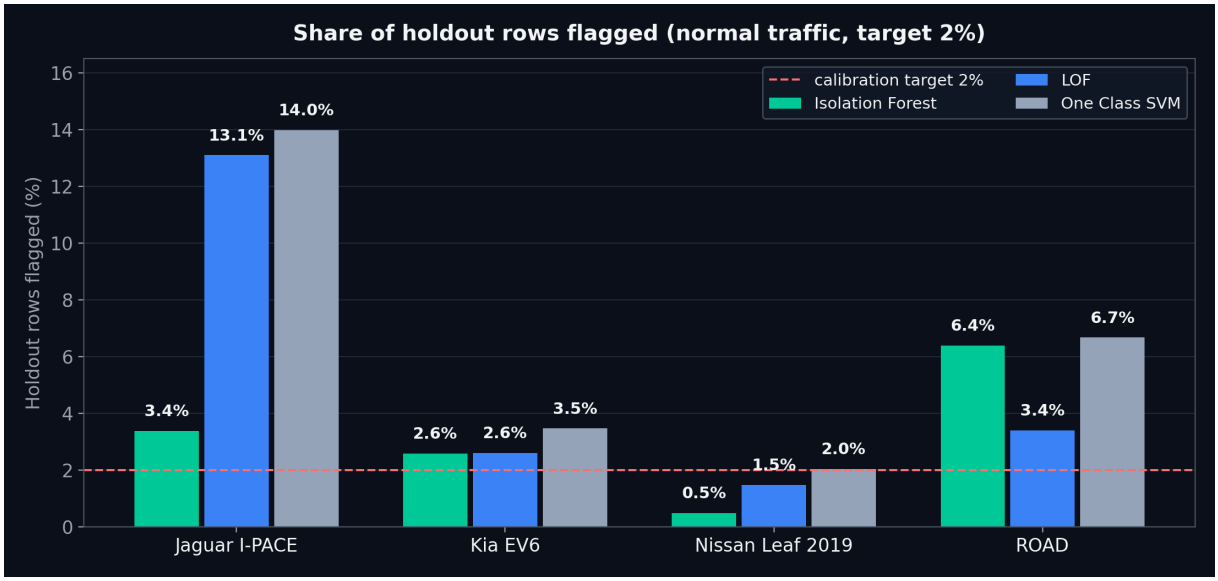


Figure 4.9: Share of the holdout rows flagged by each algorithm. The holdouts are assumed to be normal traffic, so a flagged row counts as a false alarm.

This advantage has a price (Figure 4.10). Isolation Forest is the slowest of the three: 138 seconds of training for the four vehicles against 41 and 17, and about 23 milliseconds to score one frame against 1 and 0.4. This is the model call measured in Section 4.6.5.5, and the reason the live path does not keep up with fast buses. We kept Isolation Forest because the offline analysis, the main use of the platform, needs few and even false alarms more than speed; LOF would score a live frame more than twenty times faster, at the cost of four times more false alarms on the Jaguar I-PACE.

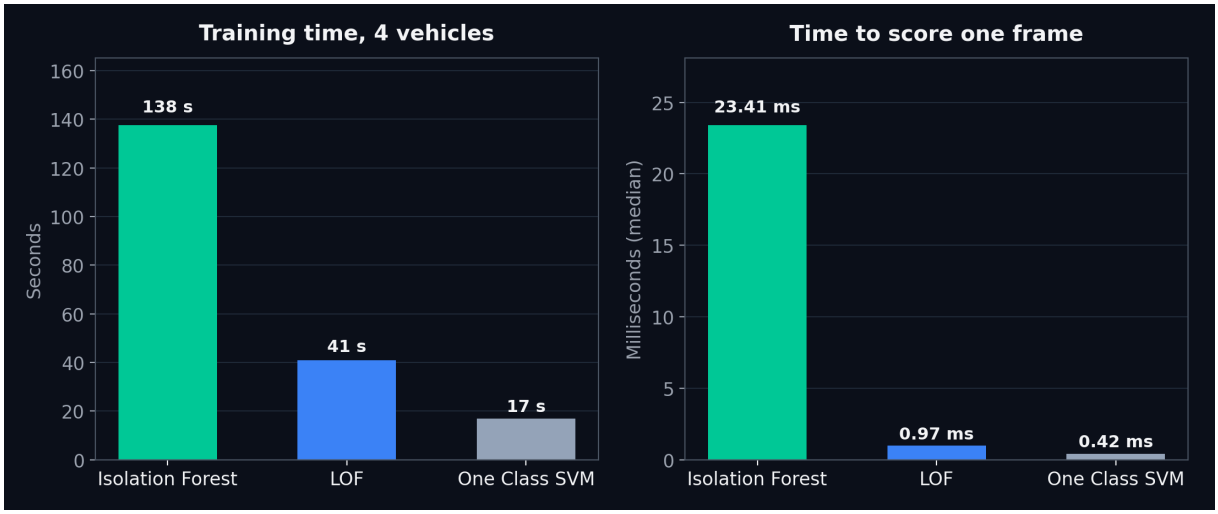


Figure 4.10: Training time for the four vehicles, and median time to score one frame.

Figure 4.11 shows what the final forests rely on. The three timing features carry about two thirds of the splits (64%) and the six payload features the rest: for most identifiers, the sending rhythm is what isolates an unusual frame first.

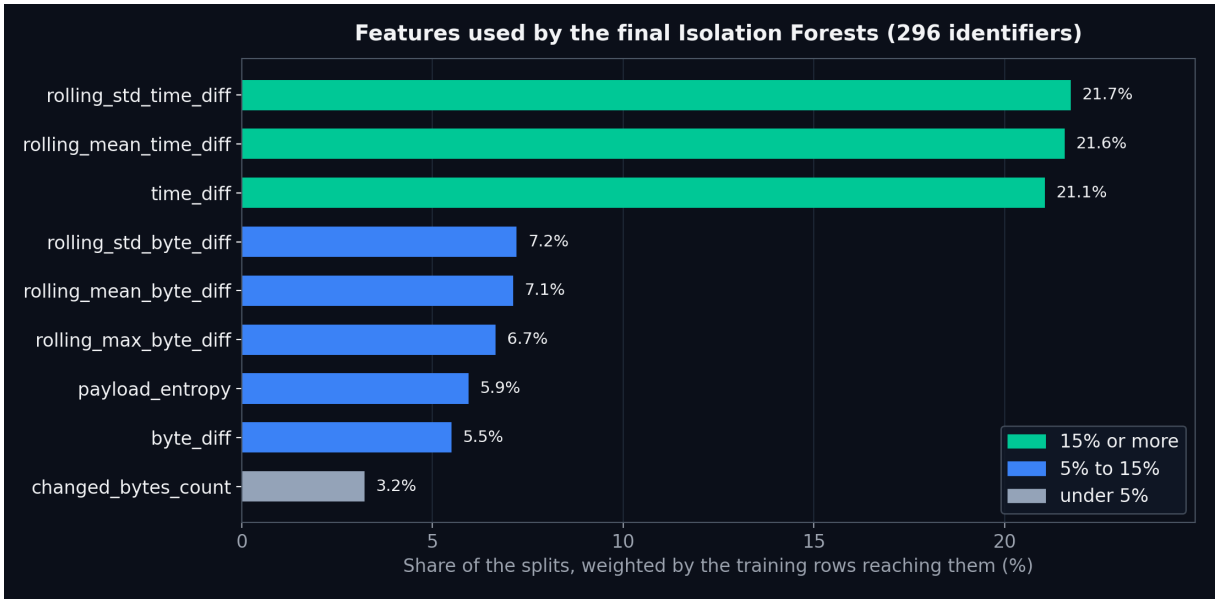


Figure 4.11: Features used by the final Isolation Forests: share of the splits, weighted by the training rows that reach them, averaged over the 296 identifier models.

4.6.6 Assessment

Four vehicles are in production, with 296 identifier models, each evaluated on a recording it never saw, with flagged rates from 0.60% to 6.09% that could each be explained. One model, the MG ZS EV, was held back because its evaluation showed that it could not be trusted. Isolation Forest was confirmed as the algorithm that flags the fewest normal rows, at the cost of a live path limited to slow buses.

4.7 Conclusion

The four iterations of the construction phase delivered the first detection pipeline and the client, validated and corrected it on a real recording, added live capture tested on a bench rig, and rebuilt the models so that each one enters production only after an evaluation on unseen data. The next chapter presents the transition phase: the final release, its tests and its deployment.

Chapter 5

Transition: Final Release and Validation

5.1 Introduction

The transition phase delivers the system to its users. This chapter presents the final release use case by use case, the test strategy and its results, the traceability of the requirements, the installation and documentation of the platform, and the scope within which it can be considered stable.

5.2 Final application tour

The screenshots below show the final release during the analysis of the Jaguar I-PACE holdout recording, a file the Jaguar I-PACE model never saw during its training. The title bar shows the short name of the application, CANvision.

- **Import log:** Every session starts on the Home screen, with the choice between a live session and an offline import (Figure 5.1). The live session stays locked until the backend runs and a USB adapter, such as an Arduino board, is plugged in. Importing a log leads to the choice of the vehicle model, with the status of each model.



Figure 5.1: Home screen: choice between a live session and an offline import.

- **Detect anomaly and review the results:** After the analysis, the Dashboard (Figure 5.2) shows the highest flagged score, the number of alerts, the share of scored frames that were flagged, the unresolved frames with their reasons, the trend of the scores and the bus monitor. For this file, the application read 50,361 frames, scored 40,477 and flagged 1,249, the 3.09% of Table 4.8, with flagged scores from 0.500 to 0.935 and 339 alerts after the merging of bursts.



Figure 5.2: Dashboard after the analysis of the Jaguar I-PACE holdout recording.

- **Describe capture:** The Dataset button opens a card that describes the recording in plain words (Figure 5.3): its number of frames, duration, identifiers and frame rate, and whether the selected model was trained on it, tested on it, or never saw it. Here, it identifies the file as a test recording the model never saw. Without a label file, it states that a flagged frame means “unusual compared with the training recordings”, not “confirmed attack”.

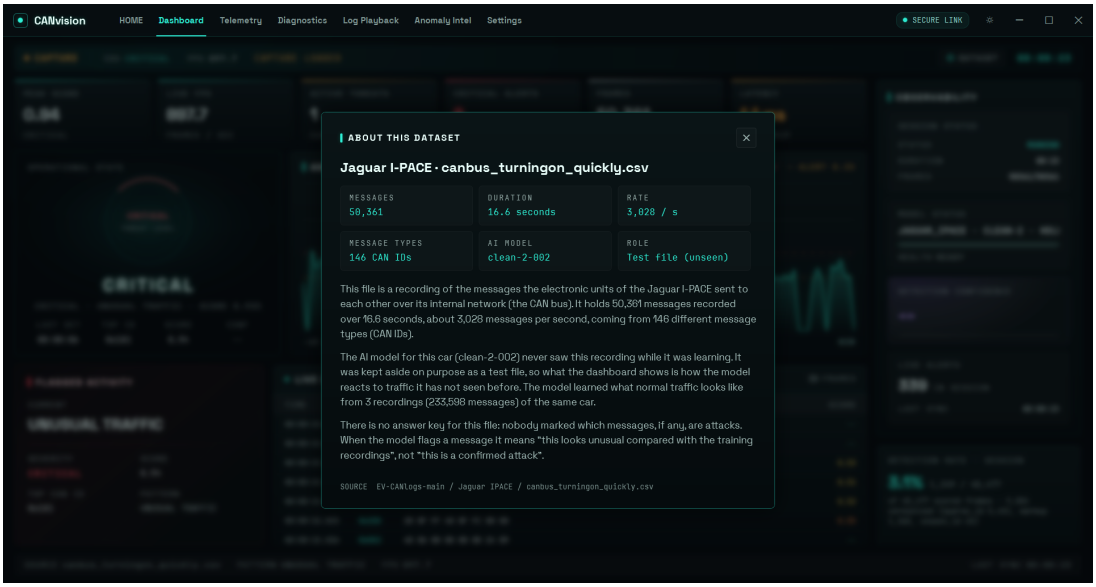


Figure 5.3: Dataset card of the Jaguar I-PACE holdout recording, open over the Dashboard.

- **Replay log:** The Log Playback screen (Figure 5.4) gives the transport controls and speed, the timeline with the flagged spans, and the detection chart with the threshold of the model.

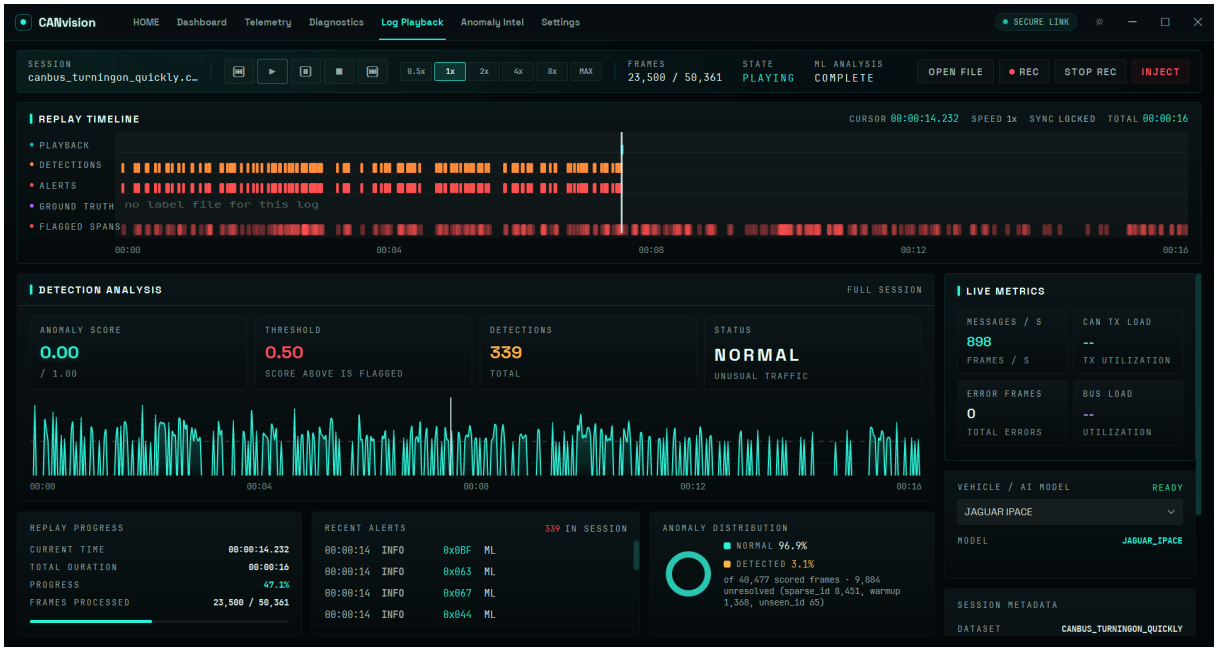


Figure 5.4: Log Playback during the replay of the Jaguar I-PACE holdout recording.

- **Review alerts and request an explanation:** Anomaly Intel (Figure 5.5) lists the alerts, shows the signals behind the selected one, and holds the explanation panel. Its alert lists display the offline severities with the labels of Table 3.1.

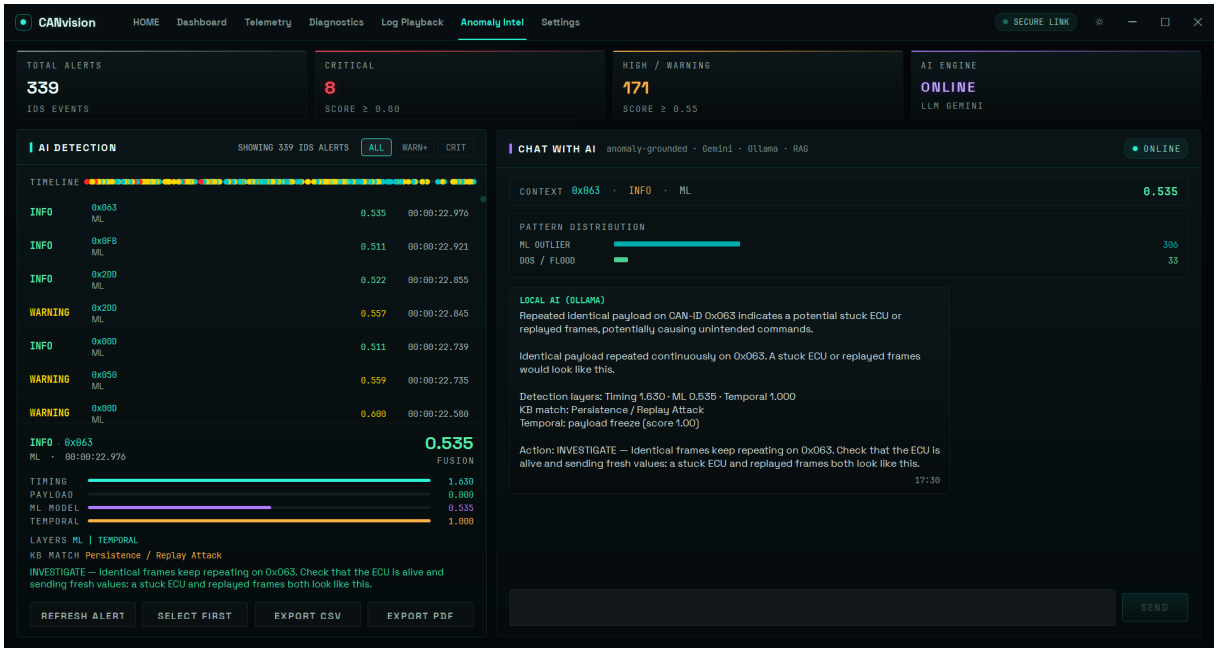
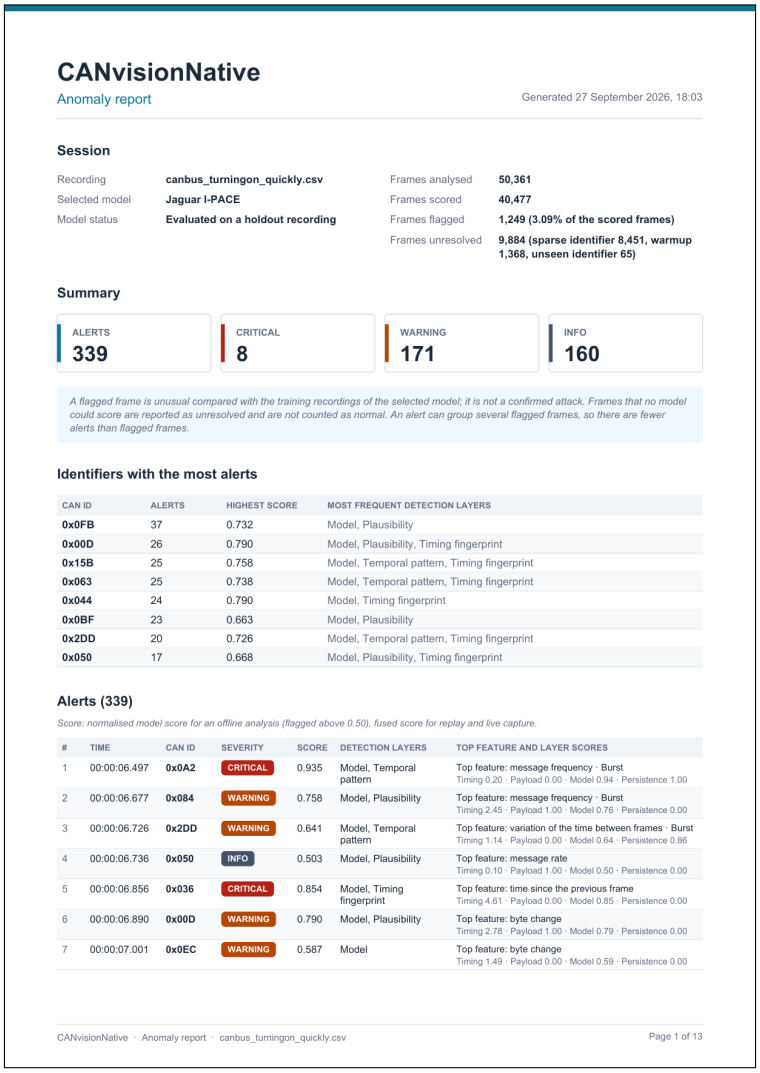


Figure 5.5: Anomaly Intel: alert list, signals of the selected alert and explanation panel.

- **Export a PDF report:** Anomaly Intel exports the alert list as a PDF report or a CSV file. The PDF report (Figure 5.6) gives the session (recording, model and its status, frames analysed, scored, flagged and unresolved), a summary by severity, the identifiers with the most alerts, the alerts in time order, and the explanations requested during the session with the engine that wrote them; for the Jaguar I-PACE analysis it has 13 pages.



CANvisionNative

Anomaly report

Generated 27 September 2026, 18:03

Session

Recording	canbus_turningon_quickly.csv	Frames analysed	50,361
Selected model	Jaguar I-PACE	Frames scored	40,477
Model status	Evaluated on a holdout recording	Frames flagged	1,249 (3.09% of the scored frames)
		Frames unresolved	9,884 (sparse identifier 8,451, warmup 1,368, unseen identifier 65)

Summary

ALERTS	CRITICAL	WARNING	INFO
339	8	171	160

A flagged frame is unusual compared with the training recordings of the selected model; it is not a confirmed attack. Frames that no model could score are reported as unresolved and are not counted as normal. An alert can group several flagged frames, so there are fewer alerts than flagged frames.

Identifiers with the most alerts

CAN ID	ALERTS	HIGHEST SCORE	MOST FREQUENT DETECTION LAYERS
0x0FB	37	0.732	Model, Plausibility
0x00D	26	0.790	Model, Plausibility, Timing fingerprint
0x15B	25	0.758	Model, Temporal pattern, Timing fingerprint
0x063	25	0.738	Model, Temporal pattern, Timing fingerprint
0x044	24	0.790	Model, Timing fingerprint
0x0BF	23	0.663	Model, Plausibility
0x2DD	20	0.726	Model, Temporal pattern, Timing fingerprint
0x050	17	0.668	Model, Plausibility, Timing fingerprint

Alerts (339)

Score: normalised model score for an offline analysis (flagged above 0.50), fused score for replay and live capture.

#	TIME	CAN ID	SEVERITY	SCORE	DETECTION LAYERS	TOP FEATURE AND LAYER SCORES
1	00:00:06.497	0x0A2	CRITICAL	0.935	Model, Temporal pattern	Top feature: message frequency · Burst Timing 0.20 · Payload 0.00 · Model 0.94 · Persistence 1.00
2	00:00:06.677	0x084	WARNING	0.758	Model, Plausibility	Top feature: message frequency · Burst Timing 2.45 · Payload 1.00 · Model 0.76 · Persistence 0.00
3	00:00:06.726	0x2DD	WARNING	0.641	Model, Temporal pattern	Top feature: variation of the time between frames · Burst Timing 1.14 · Payload 0.00 · Model 0.64 · Persistence 0.86
4	00:00:06.736	0x050	INFO	0.503	Model, Plausibility	Top feature: message rate Timing 0.10 · Payload 1.00 · Model 0.50 · Persistence 0.00
5	00:00:06.856	0x036	CRITICAL	0.854	Model, Timing fingerprint	Top feature: time since the previous frame Timing 4.61 · Payload 0.00 · Model 0.85 · Persistence 0.00
6	00:00:06.890	0x00D	WARNING	0.790	Model, Plausibility	Top feature: byte change Timing 2.78 · Payload 1.00 · Model 0.79 · Persistence 0.00
7	00:00:07.001	0x0EC	WARNING	0.587	Model	Top feature: byte change Timing 1.49 · Payload 0.00 · Model 0.59 · Persistence 0.00

CANvisionNative · Anomaly report · canbus_turningon_quickly.csv Page 1 of 13

Figure 5.6: First page of the PDF report of the Jaguar I-PACE analysis (339 alerts).

- **Export a CSV file:** The CSV file (Figure 5.7) has one row per alert, and adds the knowledge base match and the explanation of each alert to the columns of the list.

Time	CAN_ID	Severity	Pattern	Score	Reason
00:00:22.976	0x063	INFO	ML	0.535	Layers: ML + TEMPORAL · top feature msg_frequency, timing=1.630, pa...
00:00:22.921	0x0FB	INFO	ML	0.511	Layers: ML · top feature byte_diff, timing=0.250, payload=0.000, ml...
00:00:22.855	0x2DD	INFO	ML	0.522	Layers: ML · top feature byte_diff, timing=0.160, payload=0.000, ml...
00:00:22.845	0x2DD	WARNING	ML	0.557	Layers: ML · top feature msg_frequency, timing=0.360, payload=0.000...
00:00:22.739	0x00D	INFO	ML	0.511	Layers: ML + PLAUSIBILITY · top feature byte_diff, timing=0.710, pa...
00:00:22.735	0x050	WARNING	ML	0.559	Layers: ML + PLAUSIBILITY · top feature time_diff, timing=0.030, pa...
00:00:22.580	0x00D	WARNING	ML	0.600	Layers: ML + PLAUSIBILITY · top feature time_diff, timing=1.610, pa...
00:00:22.569	0x00D	WARNING	ML	0.732	Layers: ML + PLAUSIBILITY + FINGERPRINT · top feature byte_diff, ti...
00:00:22.456	0x063	INFO	ML	0.523	Layers: ML + TEMPORAL · top feature rolling_std_time_diff, timing=1...
00:00:22.416	0x063	WARNING	ML	0.552	Layers: ML + TEMPORAL · top feature msg_frequency, timing=1.740, pa...
00:00:22.399	0x098	INFO	ML	0.511	Layers: ML + FINGERPRINT · top feature payload_entropy, timing=3.06...
00:00:22.320	0x067	WARNING	ML	0.563	Layers: ML · top feature time_diff, timing=2.910, payload=0.000, ml...

Figure 5.7: First rows of the exported CSV file of the same analysis.

- **Signals of the capture:** Telemetry (Figure 5.8) shows the signals found in the capture, the live signal monitor and the activity of each identifier. No DBC file exists for the Jaguar

I-PACE, so the channels are raw payload bytes, labelled as undecoded, as required by FR12. A Diagnostics screen gathers the state of the adapter and the event log of a session.

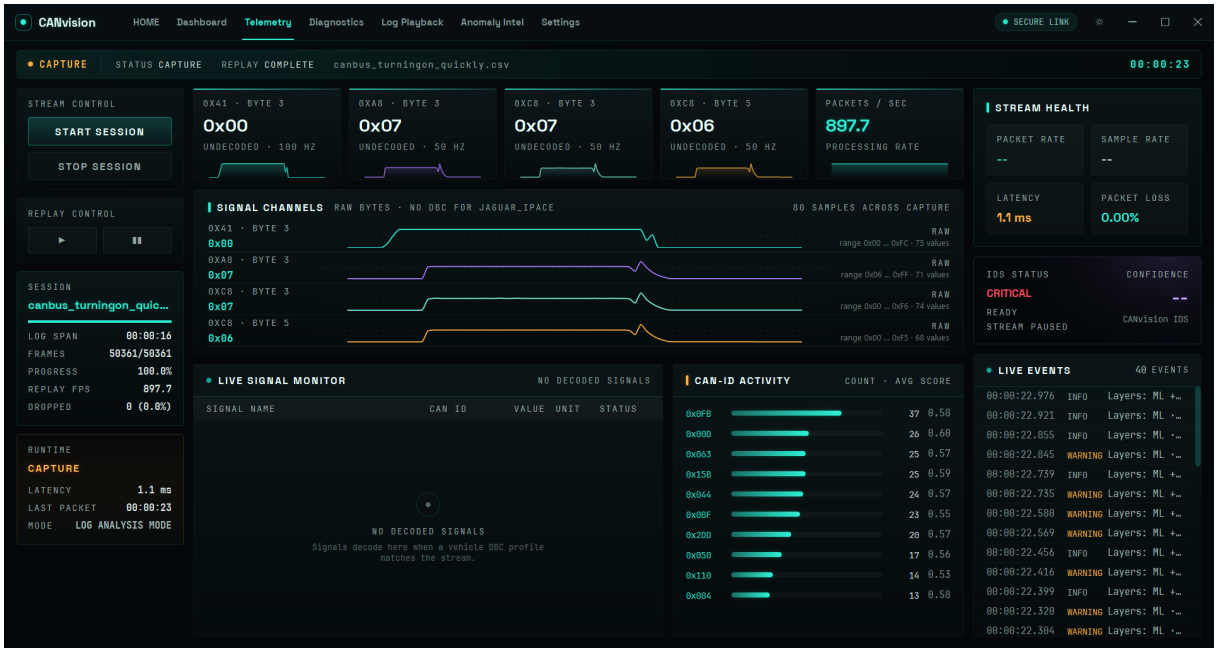


Figure 5.8: Telemetry: channels of the Jaguar I-PACE recording. No DBC file exists for this vehicle, so the channels are raw payload bytes, marked as undecoded.

- **Configure settings:** The Settings screen holds the key of the LLM provider, the hardware interface (adapter, serial port, bitrate) and an alert threshold, saved locally between sessions. In the current version, this threshold is only stored by the client: it is not sent to the backend, which scores with the values of Table 3.1.

5.3 Test strategy and results

5.3.1 Test plan

The same rule applied to every test: a claim had to be backed by reproducible evidence, a defect counted as found only once it could be triggered on demand, and as fixed only once the same test, run again, gave the expected result. Table 5.1 lists the stages of the validation over the iterations, and Table 5.2 what each kind of test covered.

Table 5.1: Stages of the validation.

Stage	**When**	**Description**
1. Interactive interface audit	Iteration 2	Exploratory testing of the running application: every screen and every control, actions in the wrong order, destructive actions, resizing and minimising during a replay.
2. Root cause analysis of the alerts	Iteration 2	Full replay of a one hour Kia EV6 recording, with every alert explained and attributed to a dominant cause.
3. Correction and new validation	Iteration 2	Each confirmed defect fixed on its own and verified again, then a full run of the corrected pipeline on the same recording.
4. Live capture on the bench rig	Iteration 3	Frames of the rig read through python-can, then through the hardware layer of the application.
5. Evaluation of the final models	Iteration 4	Each rebuilt model scored on a complete recording kept out of its training; speed measured; three algorithms compared.
6. End to end test of the release	Transition	The whole application used on a holdout recording, from import to export.

Table 5.2: Testing matrix: what was covered, and how.

Target	**Method**	**Coverage**
Desktop interface	Automation of the running client (clicks, file dialogs, keyboard) and screenshots	Navigation, import, replay controls, actions in the wrong order or repeated
Detection pipeline	Direct calls to the backend API and full replays of real recordings	Vehicle selection, scoring, reasons shown to the analyst
Parsers	Small test files per format (CSV, candump, Vector) and real recordings	Extraction of the frames, byte for byte comparison with a known good output
Final models	Scoring of complete holdout recordings, and checks of each manifest	Flagged rate on new recordings, coverage, consistency of the thresholds
Explanations	Direct API calls on two datasets from different vehicles	Correctness per alert, no mixing between sessions or datasets
Live capture	Bench rig, and the virtual bus of python-can in automated tests	Transport of the frames, decoding, alert on an unknown identifier
Export	Manual export and direct inspection of the files	Columns of the CSV file, rendering and pagination of the PDF
End to end	Full walk through the application (import, analysis, replay, explanation, export)	Integration of every layer, on real recordings

5.3.2 Automated tests

Two automated suites run after every change. The 318 backend tests (pytest) cover the parsers, the manifest checks, the model statuses and refusals, the scoring and the unresolved frames, the threshold calibration, signal discovery, the dataset description, the temporal patterns, the explanation texts and the bench rig profile. The 86 desktop tests (xUnit) cover the import of logs, the API contracts, the detection of USB devices, the dataset card, the discovered signals and the PDF report. Both suites pass, and the Release build of the client compiles without warnings.

5.3.3 End to end test of the release

Analysing the Jaguar I-PACE holdout file through the application gave the same numbers as the evaluation of Section 4.6.5.1: 50,361 frames, 40,477 scored and 1,249 flagged (Section 5.2). This test also revealed display problems, corrected before the screenshots of Section 5.2 were taken. The CSV exports wrote the scores with the decimal comma of a French Windows installation, which shifted the columns of the file; after the fix, the 339 exported rows all read back with the nine columns of the header. A bus load was shown that was in fact the processing rate of the application divided by twenty. A frequency analysis labelled an identifier sending 0.2 messages per second as a denial of service burst, because its baseline was almost zero. And some explanation texts presented a possible attack as a fact on unlabelled data.

5.3.4 Performance

The offline analysis meets NFR1: the 491,969 frames of the Nissan Leaf holdout were analysed in 187 seconds, and the Jaguar I-PACE file in about one minute in the application. The live path processes about 200 to 260 frames per second (Table 4.9), which keeps up with a slow bus such as the one of the Nissan Leaf, but not with buses that carry thousands of frames per second.

5.4 Requirements traceability

Table 5.3 links each functional requirement to its implementation and to the evidence that validates it.

Table 5.3: Requirements traceability, from requirement to implementation to validation.

Req.	Implemented in	Validated by
FR1	Common parser, Section 4.3.4.1	Fix 4, Section 4.4
FR2	Vehicle profiles and model selection, Section 4.3.4.3	Fix 1, Section 4.4
FR3	Replay engine, Section 4.3.4.2	Throughput before and after, Table 4.4
FR4	Detection paths, Section 3.6; final models, Section 4.6.3.1	Root cause analysis, Table 4.3; holdout results, Table 4.8
FR5	Severity rules, Table 3.1	Severity distribution, Figure 4.5
FR6	Explanation engine, Section 3.5.2	Fix 6, Section 4.4.4.9
FR7	Exports, Section 4.3	End to end test, Section 5.3.3
FR8	Live simulation, Section 3.8	Interactive interface audit, Section 4.4
FR9	Hardware layer, Section 4.5.4	Bench rig test, Section 4.5.5; no vehicle test
FR10	Session recording, Section 4.3	Interactive interface audit, Section 4.4
FR11	Settings, Section 5.2	Interactive interface audit, Section 4.4; the alert threshold is stored but not yet applied
FR12	Dataset card and signal discovery, Section 4.6.4	End to end test, Section 5.3.3
FR13	Model status and manifest checks, Section 4.6	Holdout evaluation, Section 4.6.5.1

Live capture has the weakest evidence: it was tested on a bench rig with synthetic traffic, not on a vehicle.

5.5 Deployment and documentation

5.5.1 Installation

The platform is installed on one Windows workstation, as shown in the deployment diagram of Figure 3.5. It needs Windows 10 or 11 with .NET Framework 4.8, Python 3.11 or later and Git; a CAN adapter is only needed for live capture. The repository contains a setup guide with the steps below and a troubleshooting section.

1. Clone the repository, create a Python virtual environment and install the dependencies of the backend:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

2. Optionally, give the explanation module a language model: enter a Google Gemini key in the Settings screen of the client, which stores it in the configuration file of the user, or set the variable `GOOGLE_API_KEY`; or install Ollama and pull a local model, which the backend uses when Gemini is not configured. Without either, the explanations are written by the rule based template.

3. Start the backend and check that it answers:

```
python -m uvicorn backend.api.server:app --host 127.0.0.1 --port 8765
curl http://127.0.0.1:8765/health
```

   The client also starts the backend by itself when it finds the Python installation.

4. Start the client, `CANvisionNative.exe`. The connection state of the backend is shown in the top right corner of every screen.

5.5.2 Model promotion

New vehicle models are produced and promoted with the commands of the training pipeline (Section 4.6.4). `plan` checks the corpus and prints the split without fitting anything; `train` fits a version into the staging folder; `validate` checks a staged version; and `promote` validates it again and moves it into production, archiving the files it replaces:

```
python -m backend.ml.training.clean_pipeline plan
python -m backend.ml.training.clean_pipeline train --version <version>
python -m backend.ml.training.clean_pipeline validate --version <version>
python -m backend.ml.training.clean_pipeline promote --version <version>
```

A held back vehicle is refused by the promotion step, and the model files in production are versioned with Git LFS.

5.5.3 User guide

Table 5.4 gives the main tasks of the analyst and how to perform them.

Table 5.4: User guide: main tasks.

Task	How
Analyse a recording	On Home, import a log file, choose the vehicle model (its status is shown next to it) and run the analysis. The results open on the Dashboard.
Check the dataset	On the Dashboard, open the Dataset card to see the size of the recording and whether the model was trained or tested on it.
Replay a recording	In Log Playback, choose a speed and play; the flagged spans and the alerts appear on the timeline.
Explain an alert	In Anomaly Intel, select an alert; the explanation panel shows the summary, the likely cause, the recommendation and the engine that wrote them.
Export the results	In Anomaly Intel, export the alert list as a PDF report or a CSV file.
Watch a live bus	Plug in a supported CAN adapter, start a live session on Home, choose the interface, port, bitrate and vehicle model, and connect.
Change the settings	In Settings, set the key of the language model, the hardware interface and the alert threshold, then save.

5.6 Stability scope and limitations

Within the scope of this validation, we consider the software stable:

- **All known defects are fixed and verified** with direct evidence.
- **Every model in production was evaluated** on a recording it never saw, and the application shows the status of every model.
- **No silent fallback remains:** Unscored frames are reported with their reason, and a model that may not be used is refused, not replaced.
- **The frame by frame paths share one engine**, and replay and offline analysis share one parser; the offline analysis turns scores into alerts differently (Table 3.1).
- **The pipeline was checked end to end** on real recordings, from import to export.

This scope has limits, which we state plainly. It is not a production, safety or security certification, and it does not cover deployment in a vehicle. Live capture was tested only on a bench rig with synthetic traffic, since no compatible vehicle was available, and the run on the physical rig has not been repeated after the last corrections. The final models were evaluated on recordings without attack labels, so we can report flagged rates but no detection rate, precision or recall; we did not evaluate the final models on the labelled attack recordings of public datasets. The live path is too slow for buses above about 260 frames per second. The weights and

thresholds of the fused score are fixed configuration values whose effect was not evaluated separately, and the alert threshold shown in the Settings screen is stored but not yet applied to scoring. The coverage of some vehicles is incomplete: on the Jaguar I-PACE, about a fifth of the frames cannot be scored, mostly because they belong to identifiers too rare to have a model. Each vehicle was evaluated on one holdout recording, which says more about that recording than about every driving situation. Finally, the explanations may cite a similar past case, and this history is kept across sessions on purpose, so an analyst who compares two unrelated vehicles may see a past case from the other vehicle.

5.7 Conclusion

The transition phase delivered a release whose screens cover every use case, checked by automated tests and by an end to end test on a recording the models never saw. Every functional requirement is traced to its implementation and its evidence, the platform comes with an installation procedure, a model promotion procedure and a user guide, and its limits are measured and stated.

General Conclusion

In this report, we presented CANvisionNative, a diagnostic and validation platform for the CAN traffic of Battery Management Systems, built through the four phases of the Unified Process, from its inception to its transition.

We reached the objectives set in Chapter 1, with one exception stated below. The project delivered a Windows desktop client and a Python backend that accept recorded logs, a live simulator and live capture from a CAN adapter. Replay, simulation and live capture share one runtime engine, and the offline analysis of a recording uses the same features and models with its own alert rules. Each vehicle has its own models, one Isolation Forest per CAN identifier, with a threshold calibrated on its training data and a manifest that records how it was built. The four models in production (Jaguar I-PACE, Kia EV6, Nissan Leaf 2019 and the ambient traffic of the ROAD dataset) were trained on 3,436,061 frames and evaluated on 1,130,500 frames of recordings they had never seen. An explanation module, separate from detection, describes each alert in plain language. The pipeline was validated on real recordings, and every defect we found was corrected and verified. The exception is live capture: it works on a bench rig of two Arduino boards, but we could not test it on a vehicle, because none was available.

For CAN validation work, CANvisionNative gathers in one tool steps that a BMS engineer usually spreads over several: importing a log in its original format, replaying it, decoding it when a DBC file exists, seeing which frames depart from the usual behaviour of their identifier, and reading why. Replay lets the same recording be run again after a change and the results be compared; live monitoring gives a view of the bus when an adapter is connected; the anomaly detection points at the identifiers worth checking first; the explanation turns a score into a text that names the dominant signal and what to check; and the exports keep a trace of the session. The platform reports anomalies, not attacks: it shows which frames it could not judge, the status of each model, and which signals are not decoded.

Besides the client and the backend, the project delivered four vehicle models evaluated on holdout recordings, each with its manifest; a training and promotion pipeline that refuses a model failing its evaluation, as for the MG ZS EV; a bench rig with its firmware and vehicle profile; test suites of 318 backend and 86 desktop tests; and an installation procedure, a model promotion procedure and a user guide. It also produced a method of validation. On the first pipeline, every one of the 15,945 alerts of a one hour Kia EV6 recording was attributed to the layer that dominated it before anything was corrected. This led to six silent defects, and correcting them reduced the alerts by a quarter and removed the saturation of the model score, without touching the detection threshold.

Looking back, each principle of the Unified Process left a mark on the project. Because the work was driven by the use cases, each iteration addressed named use cases, and the end to end test of the release walked through all of them on one recording; two of the six defects of the second iteration, the polling timer and the stale explanation, were only found when a use case was followed through the client rather than through the backend alone. Because the work was centred on the architecture, the baseline of the elaboration phase, one runtime engine for replay, simulation and live capture, paid off twice: in the third iteration, live capture only needed its transport to be built and tested, and in the fourth, every vehicle model was replaced without changing the API or the screens, apart from the display of the status of each model. Because the work was iterative, the plan could change when the evidence asked for it: the second iteration was meant to validate the first pipeline, and its findings showed that the design of the models itself was fragile, which turned the fourth iteration into a complete rebuild of the models.

The risks of the initial list (Table 2.9) also proved useful, since several of them materialised and the planned mitigations held. No vehicle was available (R1), and the shared engine and the bench rig allowed live capture to be tested anyway. The wrong model was indeed applied to a vehicle (R3, Fix 1), formats were read differently by different paths (R7, Fix 4), and an explanation could belong to another alert (R8, Fix 6); all three were found and corrected in the second iteration. A model did not generalise to its test recording (R4), and the evaluation step held it back. The live path is too slow for busy buses (R5), and this limit is measured and stated. The first pipeline did raise too many alerts (R6), 16.3% of the frames, a rate the corrections and the rebuild brought down without touching the threshold. Only the risk on the language model (R2) never materialised, since the chain of providers always produced an explanation.

The main lesson of this work is that the most costly defects were the silent ones: none of them crashed, and each one only appeared when what the system reported was compared with a fact known independently, such as a vehicle being driven or a file being valid. A holdout is only meaningful when the split is made by recording and not by frame, and a flagged rate on unlabelled data is not a detection rate. Working with real logs showed how much the data format matters, from the channel groups of an MF4 file to the diagnostic polling that shapes the timing of a recording, and the bench rig showed that a single malformed line on a serial link can matter as much as the detection code. Above all, a detection system is only as trustworthy as the evidence behind each of its numbers.

This work also has limits, detailed in Section 5.6. Live capture was tested on a bench rig, not on a vehicle. The final models were evaluated on recordings without attack labels, so we report flagged rates, from 0.60% to 6.09%, and not a detection rate, a precision or a recall. The live path scores about 200 to 260 frames per second, enough for a slow bus but not for buses that carry thousands of frames per second. Finally, each vehicle was evaluated on one holdout recording, and some vehicles have many identifiers too rare to get a model.

These limits point to the next steps. The first is to validate live capture on a real vehicle, which would close milestone M2. The second is to evaluate the final models on recordings with labelled attacks, such as those of the ROAD dataset, to measure what they detect as well as what they flag. The live path should then be made fast enough for busy buses, for example by scoring several frames at once or by using fewer trees, with each change measured on the holdout recordings.



More recordings per vehicle would give a holdout to the vehicles that have none today and a model to more of the rare identifiers.

Bibliography

[1] International Organization for Standardization, *ISO 11898-1:2015 — Road vehicles — Controller area network (CAN) — Part 1: Data link layer and physical signalling*, ISO Standard, 2015.

[2] K. Koscher et al., "Experimental security analysis of a modern automobile," in *Proceedings of the 2010 IEEE Symposium on Security and Privacy*, IEEE, 2010, pp. 447–462.

[3] S. Checkoway et al., "Comprehensive experimental analyses of automotive attack surfaces," in *Proceedings of the 20th USENIX Security Symposium*, USENIX Association, 2011.

[4] C. Miller and C. Valasek, "Remote exploitation of an unaltered passenger vehicle," IOActive, White paper, 2015, Presented at Black Hat USA 2015.

[5] E. Seo, H. M. Song, and H. K. Kim, "GIDS: GAN based intrusion detection system for in-vehicle network," in *Proceedings of the 16th Annual Conference on Privacy, Security and Trust (PST)*, IEEE, 2018.

[6] M. E. Verma, M. D. Iannacone, R. A. Bridges, S. C. Hollifield, B. Kay, and F. L. Combs, "ROAD: the real ORNL automotive dynamometer controller area network intrusion detection dataset (with a comprehensive CAN IDS dataset survey)," *arXiv preprint arXiv:2012.14600*, 2020.

[7] S.-F. Lokman, A. T. Othman, and M.-H. Abu-Bakar, "Intrusion detection system for automotive Controller Area Network (CAN) bus system: a review," *EURASIP Journal on Wireless Communications and Networking*, vol. 2019, p. 184, 2019. DOI: 10.1186/s13638-019-1484-3

[8] Vector Informatik GmbH, *CANoe*, https://www.vector.com/int/en/products/products-a-z/software/canoe/, Accessed 2026.

[9] H. M. Song, H. R. Kim, and H. K. Kim, "Intrusion detection system based on the analysis of time intervals of CAN messages for in-vehicle network," in *Proceedings of the 2016 International Conference on Information Networking (ICOIN)*, IEEE, 2016, pp. 63–68.

[10] K.-T. Cho and K. G. Shin, "Fingerprinting electronic control units for vehicle intrusion detection," in *Proceedings of the 25th USENIX Security Symposium*, USENIX Association, 2016, pp. 911–927.

[11] M. Müter and N. Asaj, "Entropy-based anomaly detection for in-vehicle networks," in *Proceedings of the 2011 IEEE Intelligent Vehicles Symposium (IV)*, IEEE, 2011, pp. 1110–1115.

[12] Linux-CAN project, *can-utils: Linux-CAN and SocketCAN user space applications*, https://github.com/linux-can/can-utils, Accessed 2026.

[13] C. Kidder, *SavvyCAN: cross platform CAN bus tool*, https://github.com/collin80/SavvyCAN, Accessed 2026.

[14] SAE International, *SAE J1979 — E/E Diagnostic Test Modes*, SAE Standard, 2017.

[15] International Organization for Standardization, *ISO 14229-1:2020 — Road vehicles — Unified diagnostic services (UDS) — Part 1: Application layer*, ISO Standard, 2020.

[16] W. W. Royce, "Managing the development of large software systems," in *Proceedings of IEEE WESCON*, 1970, pp. 1–9.

[17] K. Schwaber and J. Sutherland, *The scrum guide*, https://scrumguides.org, 2020.

[18] I. Jacobson, G. Booch, and J. Rumbaugh, *The Unified Software Development Process*. Reading, MA: Addison-Wesley, 1999.

[19] P. Roques and F. Vallée, *UML 2 en action : de l'analyse des besoins à la conception*, 4th. Paris: Eyrolles, 2007.

[20] P. Kruchten, *The Rational Unified Process: An Introduction*, 3rd. Boston, MA: Addison-Wesley, 2003.

[21] Object Management Group, *OMG unified modeling language (OMG UML), version 2.5.1*, https://www.omg.org/spec/UML/2.5.1, 2017.

[22] S. Ramírez, *FastAPI: modern, fast (high-performance) web framework for building APIs with Python*, https://fastapi.tiangolo.com, Accessed 2026.

[23] F. Pedregosa et al., "Scikit-learn: machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[24] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, IEEE, 2008, pp. 413–422.

[25] J. MacQueen, "Some methods for classification and analysis of multivariate observations," *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, no. 14, pp. 281–297, 1967.

[26] ASAM e.V., *ASAM MDF — measurement data format, version 4*, ASAM Technical Standard, 2021.

[27] python-can contributors, *python-can: controller area network support for Python*, https://python-can.readthedocs.io, Accessed 2026.

[28] CSS Electronics, *Electric vehicle data pack: CAN and UDS log files and DBC files*, https://www.csselectronics.com/pages/ev-data-pack-electric-vehicles, Accessed 2026.

[29] EV-CANlogs contributors, *EV-CANlogs: collection of CAN data from different electric vehicle projects*, https://github.com/dalathegreat/EV-CANlogs, Accessed 2026.

[30] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001.

[31] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: identifying density-based local outliers," in *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, ACM, 2000, pp. 93–104.

[32] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*, AAAI Press, 1996, pp. 226–231.

Abstract

Modern vehicles, and electric vehicles in particular, rely on the CAN bus to let their electronic control units exchange data. The CAN protocol was designed for reliability on a closed bus, not for security: it has no authentication and no encryption. In a Battery Management System (BMS), where the CAN traffic carries safety critical values such as cell voltages, state of charge and temperatures, engineers need more than a passive log of the frames.

This report presents **CANvisionNative**, a diagnostic and validation platform for the CAN traffic of Battery Management Systems, developed at Capgemini Engineering within a work and study program, alongside a full time position as a quality assurance tester of battery management software for BMW Group. The platform combines a Windows desktop client, built with WPF and the MVVM pattern, with a Python backend that exposes a REST API built on FastAPI. It imports recorded logs in the common formats, namely CSV, MF4, candump, Vector and PCAN, replays them, and can also receive live traffic from a simulator or from a CAN adapter. Each vehicle has its own machine learning models: one Isolation Forest per CAN identifier, trained on normal traffic, with a threshold calibrated on its training data. The model scores are complemented by timing, payload and persistence checks. An explanation module then describes an alert in plain language, with a language model when one is available and a rule based text otherwise; it does not take part in detection.

The project followed the Unified Process, with four construction iterations. In the second, we explained all 15,945 alerts produced by the first pipeline on a one hour Kia EV6 recording and corrected six silent defects, which reduced the alerts by a quarter without changing the detection threshold. In the fourth, we rebuilt the models with a stricter pipeline in which every model records how it was built and enters production only after an evaluation on a complete recording kept out of its training. The four models in production, trained on 3.4 million frames, flag between 0.60% and 6.09% of the frames of their test recordings, and a fifth model was held back because its evaluation showed that it could not be trusted. Since the test recordings carry no attack labels, we report these flagged rates and not detection rates. Live capture, delivered in the third iteration, was tested on a bench rig of two Arduino boards but not on a vehicle, and live scoring keeps up only with slow buses, of about 200 frames per second or less.

Keywords: Controller Area Network, Unified Process, Battery Management System, CAN diagnostics, electric vehicle, anomaly detection, Isolation Forest, holdout evaluation, explainability.

Résumé

Les véhicules modernes, et en particulier les véhicules électriques, reposent sur le bus CAN (Controller Area Network) pour permettre à leurs calculateurs d'échanger des données. Le protocole CAN a été conçu pour la fiabilité sur un bus fermé, et non pour la sécurité : il ne prévoit ni authentification ni chiffrement. Dans un système de gestion de batterie (BMS), où le trafic CAN transporte des valeurs critiques comme la tension des cellules, l'état de charge et la température, les ingénieurs ont besoin de plus qu'un simple enregistrement des trames.

Ce rapport présente **CANvisionNative**, une plateforme de diagnostic et d'aide à la validation du trafic CAN des systèmes de gestion de batterie, réalisée chez Capgemini Engineering dans le cadre d'un programme d'alternance, en parallèle d'un poste à plein temps de testeur qualité des logiciels de gestion de batterie pour le groupe BMW. La plateforme associe un client de bureau Windows, développé en WPF selon le patron MVVM, à un backend Python qui expose une API REST construite avec FastAPI. Elle importe les journaux enregistrés dans les formats courants (CSV, MF4, candump, Vector et PCAN), les rejoue, et peut aussi recevoir un trafic en direct depuis un simulateur ou un adaptateur CAN. Chaque véhicule dispose de ses propres modèles d'apprentissage automatique : une Isolation Forest par identifiant CAN, entraînée sur du trafic normal, avec un seuil calibré sur ses données d'entraînement. Les scores des modèles sont complétés par des contrôles de temporisation, de contenu et de persistance. Un module d'explication décrit ensuite une alerte en langage naturel, avec un modèle de langage lorsqu'il est disponible et un texte à base de règles sinon ; il ne participe pas à la détection.

Le projet a suivi le Processus Unifié, avec quatre itérations de construction. Au cours de la deuxième, nous avons expliqué les 15 945 alertes produites par le premier pipeline sur un enregistrement d'une heure d'une Kia EV6 et corrigé six défauts silencieux, ce qui a réduit les alertes d'un quart sans modifier le seuil de détection. Au cours de la quatrième, nous avons reconstruit les modèles avec une chaîne plus stricte : chaque modèle n'entre en production qu'après une évaluation sur un enregistrement complet exclu de son entraînement. Les quatre modèles en production, entraînés sur 3,4 millions de trames, signalent entre 0,60 % et 6,09 % des trames de leurs enregistrements de test, et un cinquième modèle a été écarté car son évaluation a montré qu'il n'était pas fiable. Ces enregistrements ne portant aucune étiquette d'attaque, nous rapportons ces taux de signalement et non des taux de détection. La capture en direct, réalisée lors de la troisième itération, a été testée sur un banc d'essai de deux cartes Arduino, mais pas sur un véhicule, et l'évaluation en direct ne suit que les bus lents, d'environ 200 trames par seconde au plus.

Mots-clés : bus CAN, Processus Unifié, système de gestion de batterie, diagnostic CAN, véhicule électrique, détection d'anomalies, Isolation Forest, évaluation sur données exclues, explicabilité.



ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn - E-mail : contact@esprit.tn

Siége Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454